Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Intelligent Information Security Systems Engineering

Cloud IAM Sentinel:

A Hybrid Framework for AWS Identity Threat Detection

A report for a Graduation Project 2 (Research) — submitted in partial fulfillment of the requirements for obtaining the Bachelor's degree in Artificial Intelligence Engineering – Intelligent Information Security Systems Engineering

Prepared by:

Gabriella Majed Haddad

Elyana Koryakus Koryakos

Khalid Moataz AL Nabulsi

Supervised by:

Dr.Christine Zenieh

Eng. Mouhamad Amjad Al Safadi

2026 – 2027

الجامعة السورية الخاصة

كلية هندسة الذكاء الاصطناعي

قسم هندسة نظم أمن المعلومات الذكية

Cloud IAM Sentinel

إطار عمل هجين للكشف عن تهديدات الهوية في خدمات AWS

تقرير مشروع تخرج 2 (بحثي) — مقدَّم استكمالاً لمتطلبات الحصول على درجة الإجازة في هندسة الذكاء الاصطناعي – هندسة نظم أمن المعلومات الذكية

إعداد:

غابريلا ماجد حداد

اليانا قرياقوس قورياكس

خالد معتز النابلسي

إشراف:

د. كريستين زينية

م. محمد أمجد الصفدي

2026 – 2027

Dedication

[……]

To our parents……

To our families ……..

Dedication

[……..]

To our parents,……...

To our families……...

Table of Contents

List of Tables

List of Figures

List of Abbreviations and Scientific Terms

Abbreviation	Term
ABAC	Attribute-Based Access Control
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
ARN	Amazon Resource Name
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
AWS	Amazon Web Services
BFS	Breadth-First Search
CIS	Center for Internet Security
CSPM	Cloud Security Posture Management
DAC	Discretionary Access Control
EC2	Elastic Compute Cloud
FAISS	Facebook AI Similarity Search
FSBP	Foundational Security Best Practices
GNN	Graph Neural Network
IaaS	Infrastructure as a Service
IAM	Identity and Access Management
JSON	JavaScript Object Notation
KMS	Key Management Service
LLM	Large Language Model
MAC	Mandatory Access Control
MFA	Multi-Factor Authentication
MITRE	Massachusetts Institute of Technology Research and Engineering

NIST	National Institute of Standards and Technology
PaaS	Platform as a Service
PDP	Policy Decision Point
PEP	Policy Enforcement Point
PFG	Permission Flow Graph
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
S3	Simple Storage Service
SaaS	Software as a Service
SCP	Service Control Policy
SP	Special Publication
STS	Security Token Service
TTP	Tactics, Techniques, and Procedures

Abstract

This research presents Cloud IAM Sentinel, a hybrid three-phase framework for AWS Identity and Access Management (IAM) security analysis that integrates static rule-based posture checking, graph-based privilege-escalation discovery, and dynamic CloudTrail correlation augmented by grounded Large Language Model (LLM) reasoning. The framework addresses a documented gap in existing cloud security tooling: rule-based Cloud Security Posture Management products examine each policy statement in isolation and miss multi-step escalation chains, while graph-based privilege-escalation analyzers treat AWS IAM permissions as binary and cannot represent Attribute-Based Access Control (ABAC) paths in which a principal first mutates a tag attribute to satisfy a downstream condition.

Phase One of the framework evaluates a catalogue of Role-Based Access Control posture rules (RBAC) and Attribute-Based Access Control (ABAC), with every rule mapped to the Center for Internet Security AWS Foundations Benchmark (CIS), the AWS Security Hub Foundational Security Best Practices, and the NIST SP 800-53 Rev. 5. Phase Two constructs a privilege-escalation graph that evaluates IAM Condition operators, applies Service Control Policy filtering, models tag-mutation actions as first-class state-transition edges, and discovers paths through bounded breadth-first search whose visited set is keyed on the joint state of node and tag values. Phase Three ingests CloudTrail management events, maps them deterministically to MITRE ATT&CK Cloud techniques, retrieves authoritative passages from a curated vector index, and produces grounded alerts through a hallucination-guarded LLM adapter.

The intended contribution is a single open framework that combines static posture analysis, graph-based privilege-escalation discovery, covering both classical RBAC escalation primitives and Attribute-Based paths in which a principal mutates a tag attribute, and dynamic CloudTrail correlation with citation-grounded Large-Language-Model reasoning, with every finding traceable to an authoritative standard. The present report documents the theoretical foundations, the architecture, and the design of each phase; the empirical evaluation of the framework against open ground-truth corpora is the subject of subsequent chapters.

Keywords: AWS Identity and Access Management; Attribute-Based Access Control; Privilege Escalation; Graph Reachability; MITRE ATT&CK Cloud; Retrieval-Augmented Generation; Hybrid Detection.

الملخّص

يُقدِّم هذا البحث Cloud IAM Sentinel، وهو إطار عمل هجين ثلاثي المراحل لتحليل أمن إدارة الهوية والوصول (IAM) في خدمات Amazon Web Services، يجمع بين فحص أمني ثابت يعتمد على القواعد، واكتشاف مسارات تصعيد الامتيازات المعتمد على الرسوم البيانية، والترابط الديناميكي مع سجلات CloudTrail مدعّم باستدلال نماذج اللغة الكبيرة اعتماداً على مصادر موثوقة. ويُعالج الإطار فجوةً موثَّقة في الأدوات الحالية لأمن السحابة، إذ تفحص أدوات إدارة الوضع الأمني السحابي القائمة على القواعد كل سياسة بمعزلٍ عن غيره فتُغفل سلاسل التصعيد متعدد الخطوات، بينما تعامل أدوات تحليل تصعيد الامتيازات القائمة على الرسوم البيانية صلاحيات IAM على أنها ثنائية، ومن ثَمَّ تعجز عن تمثيل المسارات المستندة إلى التحكم بالوصول المعتمد على السمات (ABAC) التي يقوم فيها الكيان الرئيسي أولاً بتعديل قيمة وَسْمٍ بهدف استيفاء شرطٍ لاحق.

تُقيِّم المرحلة الأولى من الإطار مجموعةً من قواعد التحكم بالوصول القائم على الأدوار (RBAC)والتحكم في الوصول القائم على السمات (ABAC)، وكل قاعدةٍ منها مُسقَطة على المعيار CIS AWS Foundations Benchmark، وممارسات AWS Security Hub، و NIST SP 800-53 Rev. 5. تبني المرحلة الثانية رسماً بيانياً لتصعيد الامتيازات بهدف تحليل معاملات الشروط في IAM، وتطبيق فلترة سياسات التحكم في الخدمات (SCP)، وتمثيل إجراءات تعديل الوسوم كخطوات أساسية لانتقال الحالة، ثم اكتشاف المسارات عبر بحثٍ متَّسع محدود العمق مع تسجيل الحالات التي تم الوصول إليها بناءً على الحالة المشتركة للعقدة وقيم الوسوم. أما المرحلة الثالثة، فتستقبل أحداث الإدارة المسجّلة في CloudTrail، وتربطها بطريقة حتمية بتقنيات مصفوفة MITRE ATT&CK السحابية، وتسترجع المقاطع الموثوقة من فهرسٍ متَّجهي مُعدٍّ مسبقاً، ثم تُنتج تنبيهاتٍ مُسنَدةً عبر مُحوِّل لنموذج لغوي كبير مزوَّدٍ بآلية للحد من الهلوسة.

تتمثل المساهمة البحثية في تقديم إطار عمل مفتوح وموحّد يجمع بين تحليل الوضع الأمني الثابت، واكتشاف تصعيد الامتيازات القائم على الرسوم البيانية، مع تغطية كلٍّ من أساسيات تصعيد الامتيازات التقليدية في RBACوالمسارات القائمة على السمات، التي يقوم فيها الكيان الرئيسي بتعديل سمة وسم معيّنة، بالإضافة إلى الربط الديناميكي لأحداث CloudTrail باستدلال نماذج اللغة الكبيرة المستند إلى مراجع موثوقة، بحيث يمكن ربط كل نتيجة مباشرةً بمعيار موثوق.

ويوثّق هذا التقرير الأسس النظرية، والبنية المعمارية، وتصميم كل مرحلة، أما التقييم التجريبي لإطار العمل بالاعتماد على مجموعات بيانات مرجعية مفتوحة وموثّقة وعالية الدقة، فهو موضوع الفصول اللاحقة.

الكلمات المفتاحية: إدارة الهوية والوصول في AWS؛ التحكم بالوصول المعتمد على السمات؛ تصعيد الامتيازات؛ قابلية الوصول في الرسوم البيانية؛ MITRE ATT&CK السحابية؛ التوليد المعزَّز بالاسترجاع؛ الكشف الهجين.

Introduction

Cloud computing dominates modern enterprise workloads, with Amazon Web Services (AWS) leading the market. The security of an AWS account is fundamentally determined by the configuration of its Identity and Access Management (IAM) policies, as IAM serves as the definitive authorization gate between principals and resources. Despite its importance, the rapid growth in cloud adoption has been accompanied by a significant rise in security incidents, the majority of which result from misconfigured identities and overly permissive policies, rather than vulnerabilities in the cloud platform itself

Three primary tool families address these risks, each with critical limitations. Cloud Security Posture Management (CSPM) tools, such as Prowler and Cloudsplaining, scan configurations against static rules but analyze policy statements in isolation, failing to detect multi-step privilege escalation. Graph-based tools, such as PMapper, model reachability but often treat permissions as binary, ignoring IAM Conditions and Attribute-Based Access Control (ABAC). Detection-as-code platforms monitor AWS CloudTrail events reactively but lack the ability to statically predict possible attack paths. These approaches do not work together, leading to fragmented security visibility and inconsistent risk-severity assessments that are not aligned with standard threat classifications.

This research proposes Cloud IAM Sentinel, a hybrid framework that integrates these domains into a single deterministic pipeline. Operating over a unified AWS inventory, the framework consists of three phases: a static posture phase that evaluates RBAC and ABAC policies against industry benchmarks, a graph phase that constructs a privilege-escalation model incorporating Deny semantics, Service Control Policies (SCPs), and IAM Conditions, with tag mutations explicitly modeled as state transitions, and a dynamic phase that maps CloudTrail events to MITRE ATT&CK techniques. To bridge the gap between raw technical data and actionable insight, the framework employs a Retrieval-Augmented Generation (RAG) pipeline that relies strictly on cited sources, ensuring that all findings are verifiable and free from hallucination.

The core research problem is the absence of an open-source framework capable of correlating multi-step ABAC privilege escalation with real-time dynamic events in a unified analysis model. This project aims to design and implement this framework, show that tag-based ABAC escalation paths can be detected statically through state modeling, and provide an open and reviewable implementation for cybersecurity researchers and practitioners.

This work is important for two main reasons. First, it presents an approach that treats IAM attributes as mutable state, revealing a type of privilege-escalation path that existing tools cannot represent. Second, by attaching authoritative citations to every finding, it ensures that the framework’s output is suitable for regulatory audit and forensic analysis.

Cloud IAM misconfiguration is a problem that no existing analysis approach fully solves, this work argues that the problem does not need to be solved by one approach alone, as long as the three main approaches, static posture analysis, graph reachability, and dynamic correlation, are combined using a common set of terms based on trusted sources. The following chapters develop this argument from its basic theoretical foundations through its architectural design and into the experimental results that supports it.

Chapter One
Theoretical Framework

1-1 Introduction

This chapter establishes the theoretical foundations of Cloud IAM Sentinel. It introduces the core security concepts behind every access-control decision, the AWS platform and its IAM service, including principal types, credentials, policies, and evaluation logic, the RBAC and ABAC access-control models and the idea that attributes can be changed, privilege escalation viewed as a limited graph reachability problem, the AWS CloudTrail along with threat classifications and security benchmarks used to label findings, and the foundations of Large Language Models and Retrieval-Augmented Generation with a citation-based grounding method that keeps every output verifiable. These primitives are carried forward into the chapters that follow.

1-2 Foundational Security Concepts

1-2-1 Authentication and Authorization

Authentication is the process of verifying that an entity is who it claims to be, whereas authorization is the process of deciding whether that authenticated entity is permitted to perform a requested action on a requested resource [16]. these two processes are distinct concerns and are typically implemented by separate mechanisms: in AWS, authentication is handled by each service verifying credentials, while authorization is performed by the IAM evaluator, which determines whether a request is allowed or denied by evaluating all applicable policies and rules [1]. The framework reasons exclusively about the authorization surface and assumes that credentials produced by the authentication layer are correctly attributed to the principal whose request they accompany.

1-2-2 The Principle of Least Privilege

The principle of least privilege requires that every principal and every process operate with the minimum set of permissions necessary to perform its function [16]. Least privilege is the central design directive of the AWS IAM service [1]. Any violation of this principle indicates the presence of excessive permissions that can be identified through analysis of access configurations. Accordingly, findings produced by the framework represent instances in which a principal holds more permissions than required within the analyzed environment.

1-2-3 Defense in Depth

Defense in depth is the practice of layering independent security controls so that the failure of any single control does not compromise the system as a whole [16]. AWS realizes defense in depth at the IAM layer through the concurrent application of identity-based policies, resource-based policies, Service Control Policies, Permission Boundaries, and session policies, all of which are included in the framework's reachability modeling [1].

1-2-4 Attack Surface and Threat Model

The attack surface of a system is the set of points at which an unauthorized principal could attempt to exercise a privilege [16]. In AWS, this mainly refers to the IAM authorization layer, where every allowed combination of principal, action, and resource forms part of the surface. The framework aims to identify cases where permissions exceed what is necessary. The threat model

assumes an internal authenticated attacker with possibly limited access, operating only through the AWS API. Attacks that bypass IAM, such as infrastructure compromise, side-channel attacks, or physical access, are not considered.

1-3 Cloud Computing and the AWS Platform

1-3-1 Definition of Cloud Computing

The National Institute of Standards and Technology defines cloud computing as a model for providing easy, on-demand network access, to a shared pool of configurable computing resources that can be rapidly allocated and released with minimal management effort [16]. The five essential characteristics listed in NIST SP 800-145 are on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [16].

1-3-2 Service Models — IaaS, PaaS, and SaaS

NIST identifies three service models for cloud computing [16]. Infrastructure-as-a-Service (IaaS) provides processing, storage, and networks on which the consumer deploys arbitrary software, including operating systems. Platform-as-a-Service (PaaS) provides a managed runtime on which the consumer deploys applications without managing the underlying infrastructure. Software-as-a-Service (SaaS) provides a complete application accessed by the consumer through a thin client. IAM itself is not one of these models, it is a cross-cutting authorization service that governs access to resources across all three. The framework targets the IaaS and PaaS surfaces of AWS, where IAM is the principal authorization control.

1-3-3 Deployment Models — Public, Private, Hybrid, and Community

The four deployment models defined by NIST [16] are public cloud (provisioned for open use by the general public), private cloud (provisioned for exclusive use by a single organization), community cloud (shared by a specific community of consumers), and hybrid cloud (a composition of two or more of the above). The framework targets public-cloud and hybrid-cloud deployments of AWS, which are the dominant operating models for AWS workloads. Both configurations expose the same AWS API surface, the IAM service, the Security Token Service, and the CloudTrail event stream, that the three phases of the framework consume, whereas private-cloud and community-cloud deployments rely on non-AWS identity systems and are out of scope.

1-3-4 The AWS Shared Responsibility Model

AWS operates under a shared responsibility model in which the provider is responsible for the security of the cloud infrastructure and the customer is responsible for the security of the workloads, configurations, and data deployed on that infrastructure [17]. Within this division of responsibility, the configuration of identities and permissions is exclusively the responsibility of the customer; this is what makes IAM the principal control surface available to the defender, and it is the surface on which the framework operates. This division of responsibility is shown in Figure 1, which shows the layers of the cloud stack governed by AWS and those governed by the customer.

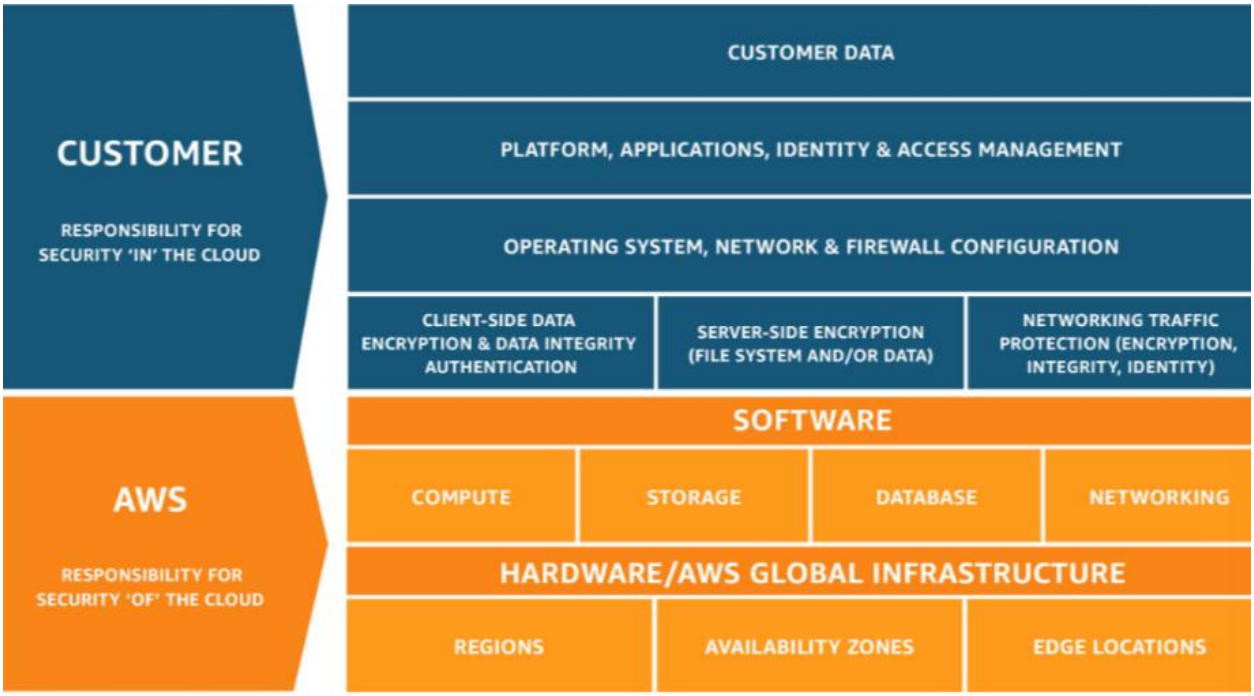

Figure 1: The AWS Shared Responsibility Model [17]

1-3-5 The AWS Account as Security Boundary

The AWS account is the fundamental security boundary of the AWS platform, every resource in AWS belongs to exactly one account, and every authorization decision is evaluated within the context of that account [1]. Multiple accounts can be grouped under an AWS Organization, and that grouping is the scope at which Service Control Policies operate by defining the maximum permissions allowed across AWS accounts, restricting actions regardless of what identity-based policies grant.

1-3-6 AWS Services

AWS services are modular cloud-based components that provide specific functionalities, such as compute, storage, networking, and identity management, within the Amazon Web Services platform [1]. Each service exposes a defined set of operations through APIs and operates within a shared infrastructure model governed by AWS security and authorization mechanisms [1].

- **AWS Identity and Access Management (IAM)**: the service that governs authentication and authorization for every principal operating within an AWS account.
- **AWS Security Token Service (STS)**: the service that issues short-lived credentials in response to AssumeRole requests, enabling cross-account access and the temporary delegation of permissions, STS is the central primitive of dynamic privilege transfer in AWS.
- **Amazon Simple Storage Service (S3)**: the object-storage service of AWS, resources are organized into buckets, and bucket-level resource-based policies determine which principals may operate on the objects they contain.

- **Amazon Elastic Compute Cloud (EC2)**: the virtual-machine service of AWS; instances may be launched with attached IAM roles, which makes EC2 a frequent participant in privilege-escalation paths through the iam:PassRole primitive.
- **AWS Key Management Service (KMS)**: the managed service for cryptographic-key creation and access, KMS key resource policies control which principals may use a key for cryptographic operations.
- **AWS Lambda**: the serverless compute service of AWS, Lambda functions execute under attached IAM roles and may be invoked through resource-based policies that grant the right to invoke to specific principals.
- **AWS CloudTrail**: the audit service that records every AWS API call as an immutable JSON event stored in an S3 bucket, CloudTrail is the source of dynamic evidence consumed by Phase Three of the framework.
- **AWS Organizations**: the service that groups AWS accounts into an organizational tree and enforces account-level governance through Service Control Policies (SCPs).
- **AWS Security Hub**: the aggregation service that consolidates security findings into the Amazon Security Finding Format (ASFF) and publishes the Foundational Security Best Practices baseline used for severity calibration in Phase One.

Together, these services form the foundational components of the AWS platform, within which identity, resource management, and service interactions are defined and controlled through a unified authorization model.

1-4 AWS Identity and Access Management (IAM)

1-4-1 The IAM Service

AWS Identity and Access Management (IAM) is the service that controls authentication and authorization for principals operating within an AWS account [1]. IAM exposes its policy and identity objects through the AWS Application Programming Interface (API), and every action performed against this API is recorded by the AWS CloudTrail service. IAM is the only authorization gate between any principal and the resources of the account, which is why IAM misconfiguration is the dominant root cause of AWS security incidents [18].

1-4-2 IAM Principal Types

A principal in IAM is an entity that can request an action on a resource. AWS recognizes six classes of principal that the framework models: the root user, IAM users, IAM groups, IAM roles, federated identities, and service principals (including service-linked roles) [1]. Each class is defined in the following subsections.

1-4-2-1 The Root User

The root user is the identity created when an AWS account is first opened [1]. It has unrestricted access to every resource in the account, cannot have its permissions reduced by any IAM policy, and cannot be deleted. AWS recommends that the root user be used only for the small number of account-management tasks that require it, and that all routine work be performed under IAM users

or IAM roles. The framework reports any unusual use of the root user under its Phase One posture rules and flags it as a high-severity finding.

1-4-2-2 IAM Users

An IAM user is a long-lived identity within an AWS account, intended for a specific human or for a specific application [1]. An IAM user is created with a unique name and is associated with different types of credentials. An IAM user can be a member of one or more IAM groups and can have any number of identity-based policies attached, in addition to inline policies embedded directly within the user object.

1-4-2-3 IAM Groups

An IAM group is a logical container of IAM users that simplifies permission administration [1]. Groups are not principals, meaning they cannot make requests, but the policies attached to a group apply to all users who are members of it. Groups cannot be nested, and an IAM user can belong to multiple groups at the same time.

1-4-2-4 IAM Roles

An IAM role is an identity within an AWS account that does not have long-lived credentials of its own, rather, the role is assumed by a trusted principal through a call to the AWS Security Token Service [1]. Each role has a trust policy that defines which principals are allowed to assume it, along with one or more identity-based policies that specify the permissions available during the assumed-role session. Roles are the standard mechanism for granting temporary credentials to compute instances, AWS services, and cross-account users.

1-4-2-5 Federated Identities

A federated identity is a principal authenticated by an external identity provider, such as an enterprise SAML provider, an OpenID Connect provider, or AWS IAM Identity Center, and mapped to an IAM role through a trust policy [1]. The framework treats a federated session as a role assumption for graph analysis, the session inherits the permissions of the assumed role, and is governed by all relevant policy types.

1-4-2-6 Service Principals and Service-Linked Roles

A service principal is an AWS service that may itself act as a principal in calls to other AWS services [1]. For example, when an Amazon Elastic Compute Cloud (EC2) instance is launched with an attached IAM role, the EC2 service acts as the service principal that assumes the role on behalf of the instance. A service-linked role is a special type of role created and managed by an AWS service, which uses it to perform actions on behalf of the customer, these roles cannot be modified by the customer. The framework models both types of service principals in the privilege graph, since the **iam:PassRole** permission allows services to assume roles defined within the account.

1-4-3 Authentication Credentials

AWS recognizes two broad classes of credential [1]. Long-lived credentials are bound to an IAM user and consist of a console password (used by humans) and access keys (used by programs), each access key is the pair of an Access Key Identifier and a Secret Access Key. Temporary session

credentials are issued by the AWS Security Token Service (STS) after a successful AssumeRole request and consist of an Access Key Identifier, a Secret Access Key, and a Session Token, all of which expire after a configurable duration. The framework treats the creation of any new credential, whether long-lived through iam:CreateAccessKey or temporary through sts:AssumeRole, as a graph edge that transfers the resulting permissions to the target identity.

1-4-4 Multi-Factor Authentication

Multi-Factor Authentication (MFA) is the requirement that a principal present an additional factor, typically a time-based one-time password generated by a virtual or hardware device, in addition to its primary credential [1]. Several Phase One posture rules in the framework verify the configuration of MFA on the root user and on IAM users with console access, because the absence of MFA is a high-severity violation under the CIS AWS Foundations Benchmark [12].

1-5 IAM Policies

1-5-1 Policy Documents and JSON Structure

An IAM policy is a JavaScript Object Notation (JSON) document that specifies a set of permitted or denied actions on a set of resources under a set of conditions [1]. A policy document is a top-level JSON object that includes an optional Version element and a required Statement element, which may contain either a single statement or a list of statements. Each statement includes the relevant elements that define permissions and conditions.

1-5-2 Managed Policies and Inline Policies

AWS distinguishes managed policies from inline policies [1]. A managed policy is a stand-alone, versioned policy that can be attached to many identities, managed policies are subdivided into AWS-managed (defined by AWS and read-only) and customer-managed (defined by the customer). An inline policy is a policy embedded directly within a single identity or resource and shares its lifecycle. The framework parses both classes of policy and keeps them separate in the inventory because inline policies require different traversal rules during graph construction.

1-5-3 Identity-Based, Resource-Based, and Trust Policies

Three sub-classes of policy participate in IAM authorization [1]. An identity-based policy is attached to an IAM user, group, or role and grants permissions to that identity. A resource-based policy is attached to a resource, such as an Amazon Simple Storage Service (S3) bucket, an AWS Key Management Service (KMS) key, or an AWS Lambda function, and specifies which principals may operate on it. A trust policy is a special form of resource-based policy linked to an IAM role, it specifies which principals are permitted to assume the role through the Security Token Service. The framework parses all three sub-classes and treats them as supporting inputs to graph construction.

1-5-4 Policy Elements

Each statement within an IAM policy carries the following elements [1]. The Effect element takes one of two values, Allow or Deny, and determines whether the statement contributes a positive or a negative authorization. The Action element enumerates the AWS API operations to which the

statement applies, expressed as service-prefixed identifiers such as iam:PassRole or s3:GetObject. The Resource element lists the Amazon Resource Names (ARNs) of the resources to which the statement applies, with wildcards permitted. The Principal element appears only in resource-based policies and identifies the principals to which the policy grants access. The Condition element is an optional Boolean expression evaluated at request time over a set of pre-defined and policy-defined keys, and it is the element that introduces ABAC semantics into IAM.

1-5-5 NotAction and NotResource

The negative forms NotAction and NotResource are alternative elements that match all actions or resources that are not explicitly listed in the policy [1]. The framework converts these forms during policy parsing into the equivalent positive forms when feasible, because analyzing "all actions except listed ones" is more reliable when compared against the full AWS action list.

1-5-6 Amazon Resource Names (ARNs)

An Amazon Resource Name (ARN) uniquely identifies an AWS resource. The general syntax is arn:partition:service:region:account-id:resource-id, where partition is aws for the standard regions, service is the service name (for example s3 or iam), region is the region in which the resource resides (empty for global resources such as IAM), account-id is the twelve-digit AWS account identifier, and resource-id is a service-specific identifier [1]. The framework matches ARNs in policy statements against the ARNs of inventory resources during graph construction.

1-5-7 Action Wildcards

Both Action and Resource elements support the star wildcard (*), which matches any sequence of characters [1]. A statement that grants Action: "*" over Resource: "*" grants administrative access and is treated by the framework as a critical-severity finding, a statement that grants iam:* over a specific role is similarly treated as a high-severity violation because it permits the holder to attach arbitrary policies to that role.

1-5-8 Policy Variables

Policy variables are placeholders such as ${aws:username} and ${aws:userid} that are replaced at request time with the corresponding attribute of the requesting principal [1]. Policy variables allow patterns like limiting a user to their own S3 folder, the framework evaluates policy variables symbolically during graph construction.

1-6 Policy Evaluation Logic

1-6-1 The Six Policy Classes

AWS IAM defines six classes of policy that participate in any single authorization decision [1], identity-based policies, resource-based policies, Service Control Policies (SCPs), Permission Boundaries, session policies, and access control lists (ACLs). The framework supports the first five classes, ACLs, supported by a small set of legacy services, are out of scope because the project targets modern IAM-based authorization only.

1-6-2 The Deterministic Evaluation Algorithm

When a principal issues a request, AWS IAM evaluates all applicable policies according to a deterministic algorithm based on two main rules [1]. First, an explicit Deny always overrides any Allow, this deny-overrides property is replicated by the framework when determining whether a graph edge is valid. Second, if there is no Allow, access is denied by default, this is known as the closed-world assumption in IAM and forms the basis of least-privilege enforcement.

1-6-3 Explicit Deny, Explicit Allow, and Implicit Deny

An explicit Deny is any statement whose Effect is Deny and whose Action / Resource / Principal / Condition all match the request, an explicit Allow is a statement with Effect set to Allow, an implicit Deny is the absence of any matching statement [1]. The framework records the source (policy class, policy name, statement identifier) of every Deny and Allow that affects the final decision, so that any rejected escalation path can be explained by reference to the specific statement that closed it.

1-6-4 AWS Security Token Service and Assume Role

The AWS Security Token Service (STS) issues temporary credentials in response to a successful AssumeRole request [1]. ach IAM role has a trust policy that defines which principals are allowed to assume it, an AssumeRole request succeeds only if the caller is permitted by both the trust policy and the caller's identity-based policy. The temporary credentials provided by STS inherit the permissions of the assumed role and expire after a configurable period. AssumeRole is the primary mechanism for cross-account access in AWS and is frequently involved in privilege escalation, since a successful role assumption grants the full set of permissions associated with the target role [19].

1-6-5 Service Control Policies (SCPs)

A Service Control Policy (SCP) is an organization-level control that defines the maximum permissions available to accounts within an AWS Organization [1]. SCPs act as filters and do not grant permissions, if a request is denied by any SCP in the hierarchy from the organization root to the target account, it is denied regardless of any Allow in identity-based or resource-based policies. The framework treats SCPs as filters applied to permissions during graph building.

1-6-6 Permission Boundaries

A Permission Boundary is an attached managed policy that defines the maximum permissions an identity-based policy can grant to a principal [1]. Permission Boundaries act as the identity-level equivalent of SCPs, they restrict the effective permissions of a single principal regardless of what its attached policies allow. The framework models Permission Boundaries as a per-principal constraint applied to permissions when constructing each outgoing edge from that principal in the graph.

1-6-7 Session Policies

A Session Policy is an inline policy provided during an AssumeRole call to further restrict the permissions of the resulting session beyond those granted by the role [1]. Session policies are temporary and apply only to a specific request, the framework models them as per-edge filters that apply only to AssumeRole edges in the graph.

1-7 Access Control Models

1-7-1 DAC, MAC, and RBAC Families

Access-control research recognizes three classical families of policy: Discretionary Access Control (DAC), in which the owner of an object decides who may access it, Mandatory Access Control (MAC), in which a system-wide policy based on labels prevents owners from granting access in a way that breaks the labeling rules, and Role-Based Access Control (RBAC), in which permissions are associated with roles rather than with individual users [2]. The framework is concerned with RBAC and ABAC, because these are the two access-control families realized in AWS IAM.

1-7-2 Role-Based Access Control

Role-Based Access Control (RBAC) is the model in which permissions are assigned to roles, and roles are assigned to users, so that authorization decisions are based on the role membership of the requester rather than on individual identity [2]. The four core components of the RBAC reference model are users (U), roles (R), permissions (P), and sessions (S), together with two assignment relations: user-assignment $UA \subseteq U \times R$ and permission-assignment $PA \subseteq P \times R$ [2]. Optional extensions include role hierarchies, Static Separation of Duty (SSD) constraints that forbid simultaneous membership in conflicting roles, and Dynamic Separation of Duty (DSD) constraints that prevent simultaneous activation of conflicting roles within a session [2]. AWS IAM realizes RBAC through the assume-role primitive of the Security Token Service and through the attachment of managed and inline policies to user, group, and role identities [1].

1-7-3 Attribute-Based Access Control

Attribute-Based Access Control (ABAC) is an access control model in which authorization decisions are determined by evaluating attributes associated with the subject, object, action, and environment against policies expressed as Boolean functions over those attributes [3]. The basic ABAC scenario is illustrated in Figure 2: a subject requests access to an object, and the access-control system evaluates the policy rules against the subject attributes, the object attributes, and environmental conditions, the subject is granted access if and only if the policy evaluates to permit. ABAC generalizes Access Control Lists (ACLs) and RBAC as special cases based on identity and role attributes, respectively [3]. The functional architecture of an ABAC mechanism includes a Policy Decision Point (PDP), which evaluates policies, and a Policy Enforcement Point (PEP), which intercepts requests and enforces the decision returned by the PDP, as shown in Figure 3, an enterprise-scale deployment of these components is illustrated in Figure 4.

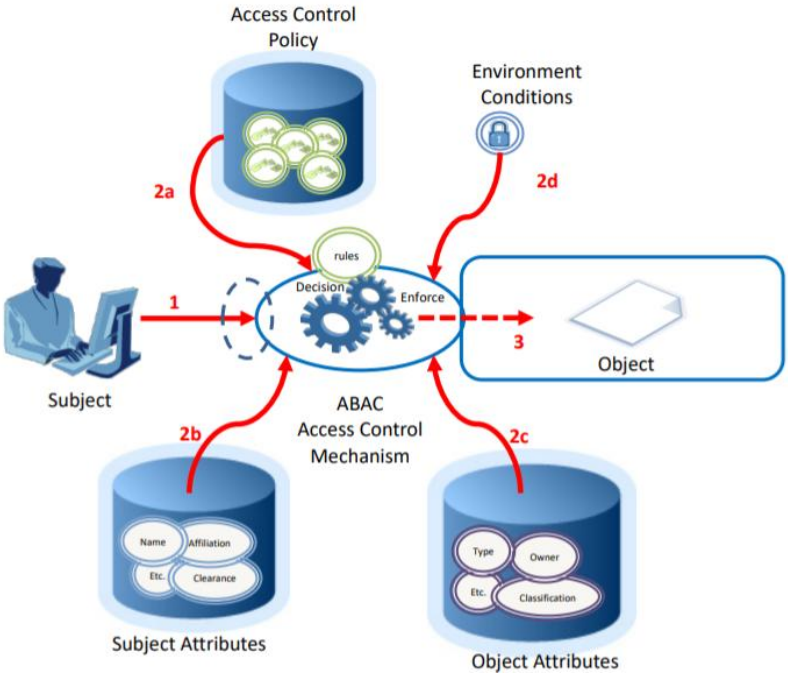

Figure 2: Basic ABAC Scenario — NIST SP 800-162 [3]

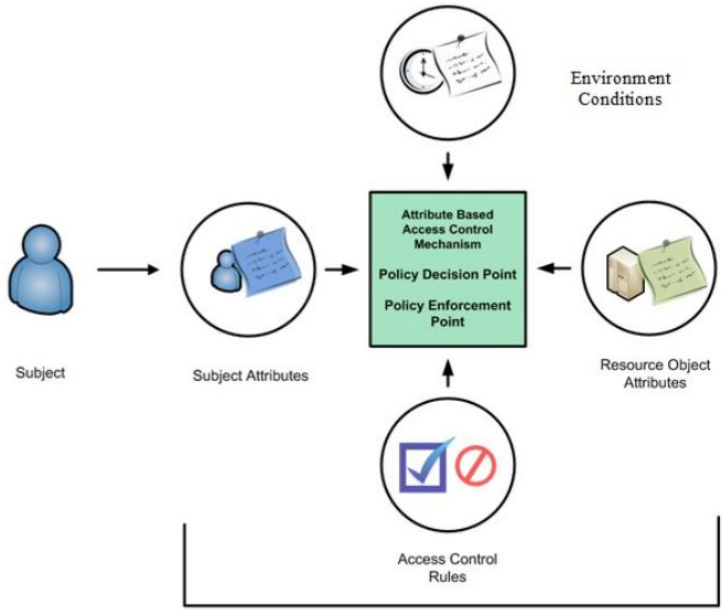

Figure 3: Core ABAC Mechanisms — Policy Decision Point and Policy Enforcement Point — NIST SP 800-162 [3]

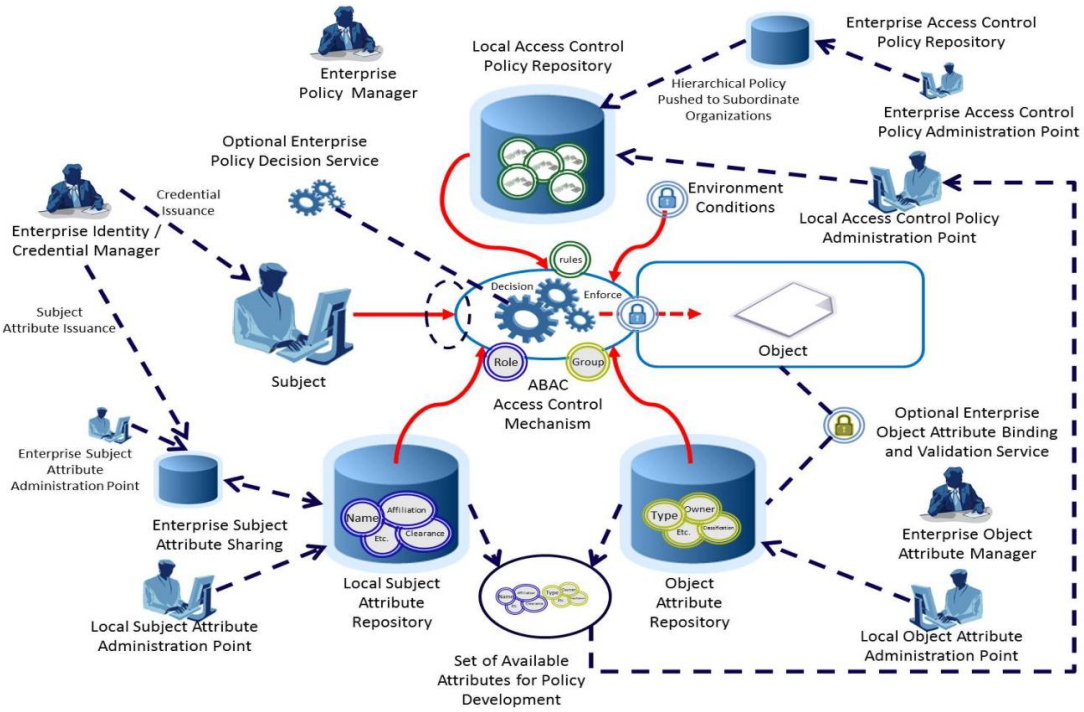

Figure 4: Enterprise ABAC Scenario — NIST SP 800-162 [3]

1-7-4 Attribute Mutability

Because attributes can change over time, ABAC introduces the formal notion of attribute mutability, defined as the property of an attribute whose value may change as a side-effect of permitted actions [5]. NIST SP 800-205 distinguishes between immutable and mutable attributes and notes that the correctness of static analysis of an attribute-based system depends on when the attribute is evaluated [5]. This is the property that the Phase Two graph engine treats explicitly, a tag-mutation edge changes the value of an attribute, and any condition that later uses that attribute must be re-evaluated against the new state.

1-7-5 Implementation of RBAC and ABAC in AWS IAM

AWS IAM implements RBAC through the use of roles and policy attachments, and supports ABAC through Condition keys and Condition operators [4]. The Condition keys most relevant to the framework are aws:PrincipalTag/<key>, aws:ResourceTag/<key>, aws:RequestTag/<key>, and aws:TagKeys, all of which reference tag attributes attached to identities or resources [4]. The logical components involved in an attribute-based authorization decision, including subject attributes, object attributes, contextual information, authentication policies, authorization policies, and the corresponding decision and enforcement points, are illustrated in Figure 5.

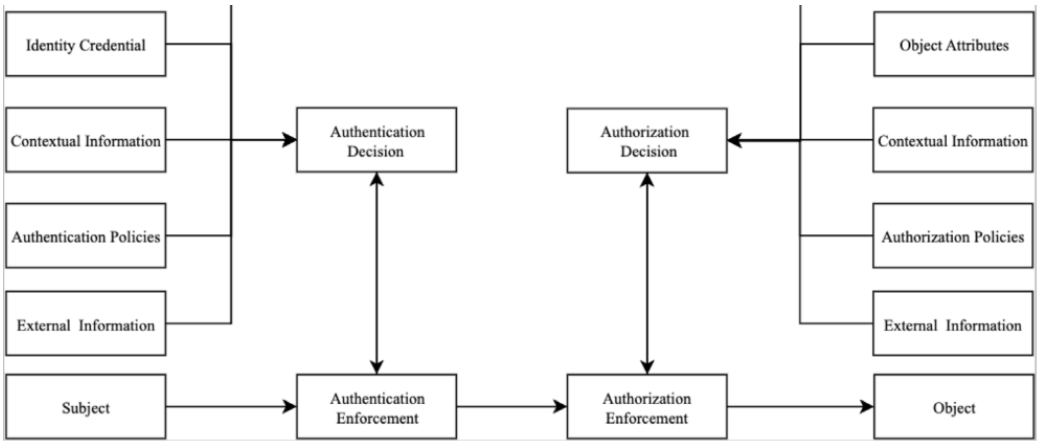

Figure 5: Attribute-Based Access Management — Logical Components and Authentication / Authorization Decision [18]

1-8 IAM Privilege Escalation

1-8-1 Definition

A privilege-escalation path in IAM is a sequence of permitted actions that allows a principal to obtain a set of permissions strictly greater than its initial set [6], [19]. Such paths arise either from misconfigured policies that grant transitive privilege-granting permissions or from logical compositions of individually benign permissions, in both cases, the resulting effective permissions available to the attacker exceed those granted by any individual policy in the inventory.

1-8-2 Common Escalation Primitives

The escalation primitives most frequently encountered in published IAM vulnerability catalogues include iam:PassRole (which permits the caller to associate an IAM role with a service such as Amazon Elastic Compute Cloud, inheriting the role's permissions through the service), sts:AssumeRole (which permits the caller to obtain temporary credentials of a target role), iam:CreateAccessKey (which allows the caller to create long-term credentials for a target user), iam:UpdateAssumeRolePolicy (which permits the caller to modify a role's trust policy and add itself as a permitted assumer), iam:AttachUserPolicy and iam:AttachRolePolicy (which permit the caller to extend a target identity's permission set with managed policies), and iam:PutUserPolicy and iam:PutRolePolicy (the inline analogues of the previous two) [6], [19].

1-8-3 Graph-Based Reachability Analysis

The privilege-escalation discovery problem can be formulated as a reachability problem on a directed graph where nodes are principals, roles, and resources, and whose edges represent permitted privilege-granting actions [19]. Hu et al. formalize this structure as a Permission Flow Graph (PFG), illustrated in Figure 6 for a well-known 2019 IAM privilege-escalation incident, where an attacker held permissions over a service principal that could assume a role, which in turn could assume an administrator-equivalent role, the figure further illustrates the transformation from a raw permission graph to an analyzed model, revealing hidden privilege-escalation paths that emerge from the composition of individually benign permissions [19].

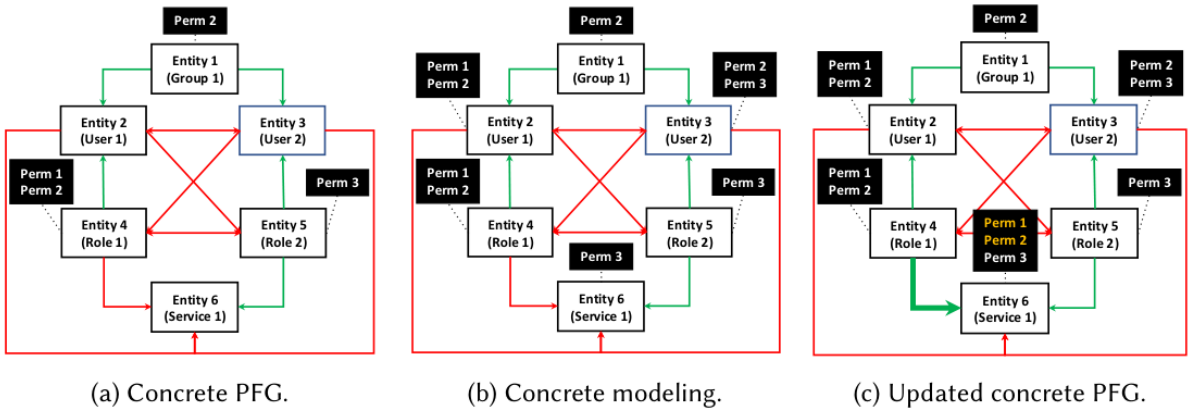

Figure 6: Permission Flow Graph (PFG) examples for the 2019 IAM privilege-escalation incident [19]

1-8-4 Tag Mutation as State Transition

A class of privilege-escalation paths cannot be represented in a permission-only graph, because the satisfaction of an edge may depend on the value of a tag that a principal can modify prior to traversing that edge [5]. NIST SP 800-205 formalizes this property as attribute mutability and emphasizes that any analysis of an attribute-based system must account for the time at which an attribute is evaluated [5]. Tag-mutation actions—including iam:TagUser, iam:UntagUser, iam:TagRole, iam:UntagRole, ec2:CreateTags, ec2:DeleteTags, s3:PutBucketTagging, and s3:DeleteBucketTagging—can be modeled as first-class graph edges representing state transitions in the attributes of principals or resources [9].

1-8-5 Bounded Breadth-First Search over Joint State

Reachability over the augmented graph is computed using a bounded breadth-first search (BFS) starting from each principal, with two adjustments. First, the visited set is indexed not only by the node, but by the joint state (node, tag-state-hash), allowing the same node to be revisited under different attribute configurations. Second, the search depth is bounded by a configurable parameter k, in order to maintain tractability on large inventories while preserving coverage of previously documented escalation paths in [6] and [7], which are typically short in practice and often require no more than a small number of steps (e.g., ≤ 3) in publicly available datasets.

1-9 Dynamic Detection through CloudTrail

1-9-1 The CloudTrail Service

AWS CloudTrail is the service that records AWS API activity as immutable JSON events stored in an Amazon S3 bucket and optionally delivered to CloudWatch Logs and EventBridge [10]. Each CloudTrail event contains information about the caller identity (userIdentity), the invoked action (eventName, eventSource), the affected resources (resources), the request context (sourceIPAddress, userAgent), and the associated request and response parameters. CloudTrail provides an authoritative record of API activity and authorization outcomes within an AWS account.

1-9-2 Management Events and Data Events

CloudTrail distinguishes between two classes of event [10]. Management events record control-plane operations such as CreateUser, AssumeRole, AttachRolePolicy, and PutBucketPolicy, and are recorded by default. Data events record data-plane operations such as s3:GetObject and lambda:Invoke, and require an explicit configuration. Privilege-escalation activity in AWS IAM occurs primarily in the control plane, making management events the relevant class for its analysis.

1-9-3 The CloudTrail Event Schema

Each CloudTrail management event is a JSON document that includes the fields such as eventVersion, userIdentity (with subfields type, principalId, arn, accountId, and sessionContext), eventTime, eventSource, eventName, awsRegion, sourceIPAddress, userAgent, requestParameters, responseElements, requestID, eventID, and resources [10]. These fields provide a structured representation of API activity that can be systematically mapped to higher-level security models, such as threat taxonomies.

1-10 Adversary Tactics: MITRE ATT&CK

1-10-1 The MITRE ATT&CK Framework

MITRE ATT&CK is a publicly maintained knowledge base of adversary behavior observed in real-world intrusions, organized into matrices for distinct technology domains [11]. It provides a standardized vocabulary for describing adversary tactics and techniques, enabling consistent classification, analysis, and communication of security events across the cybersecurity community.

1-10-2 Tactics, Techniques, Sub-Techniques, and Procedures

ATT&CK organizes adversary behavior into a four-level taxonomy [11]. A tactic represents a high-level adversary objective (e.g., Privilege Escalation, Credential Access, or Defense Evasion). A technique describes a method by which an adversary achieves a tactic (e.g., T1078 Valid Accounts). A sub-technique provides a more specific instance of a technique (e.g., T1078.004 Valid Accounts: Cloud Accounts). A procedure refers to a real-world implementation of a technique by a specific threat actor.

1-10-3 The ATT&CK for Cloud (IaaS) Matrix

The ATT&CK for Cloud matrix and its IaaS sub-matrix enumerate techniques relevant to cloud platforms such as AWS, Azure, and GCP [11]. Among these, several techniques are particularly relevant to cloud identity and access management scenarios, including T1078.004 Valid Accounts: Cloud Accounts, T1098.001 Account Manipulation: Additional Cloud Credentials, T1098.003 Account Manipulation: Additional Cloud Roles, T1136.003 Create Account: Cloud Account, T1530 Data from Cloud Storage, T1552.005 Unsecured Credentials: Cloud Instance Metadata API, and T1562.008 Impair Defenses: Disable or Modify Cloud Logs [11].

1-11 Posture Benchmarks

1-11-1 The CIS AWS Foundations Benchmark

The Center for Internet Security (CIS) AWS Foundations Benchmark is a community-developed configuration baseline that prescribes recommended security control settings for AWS accounts [12]. Each control is mapped to a CIS Critical Security Control identifier, an applicable Profile Level (Level 1 for general production environments, Level 2 for environments requiring enhanced security), as well as corresponding audit and remediation procedures [12]. The benchmark provides a standardized reference for evaluating the security posture of AWS environments and enables findings to be traced to an authoritative baseline.

1-11-2 AWS Security Hub Foundational Security Best Practices

AWS Security Hub provides the Foundational Security Best Practices (FSBP) standard, a vendor-defined catalogue of security controls that complements the CIS AWS Foundations Benchmark and assigns a default severity level using the Amazon Security Finding Format (ASFF) [1]. The FSBP standard serves as a practical reference for continuous security monitoring and aligns findings with a consistent severity model.

1-11-3 NIST SP 800-53 Revision 5 and Control Structure

NIST Special Publication 800-53 Revision 5 is the United States Federal catalogue of security and privacy controls for information systems and organizations [15]. Each control in the catalogue follows a standardized structure consisting of a base control section, a discussion section, related-controls section, control-enhancements, and references [15]. The framework maps every rule to the most specific NIST 800-53 control family identifier (e.g., AC-6 Least Privilege) so that findings can be consumed by compliance pipelines that require NIST traceability.

1-12 Large Language Models and Retrieval-Augmented Generation

1-12-1 Large Language Models

A Large Language Model (LLM) is a transformer neural network trained on large textual corpora that produces natural-language output conditioned on an input prompt. LLMs can be specialized for particular domains or trained for general-purpose use, and they are commonly deployed through local or remote inference systems. Their ability to process structured and unstructured inputs makes them suitable for tasks such as reasoning, summarization, and decision support.

1-12-2 The Hallucination Problem

Free-form generation by a Large Language Model (LLM) can produce statements that are not supported by any source, a phenomenon known as hallucination. In a security-sensitive contexts, hallucinated content is problematic because it can mislead the analysts, fabricate a non-existent identifier, or incorrectly attribute actions to entities that did not perform them [21]. Approaches to mitigating hallucination commonly include techniques such as retrieval-augmented generation and grounding outputs in verifiable sources.

1-12-3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is an inference-time architecture in which a retriever first selects a set of passages from an external knowledge corpus that are most relevant to the input query, and a generator then conditions its output on both the retrieved passages in addition to the original input [13]. This architecture enables the incorporation of external, up-to-date, and domain-specific knowledge at generation time, improving the factual accuracy of the generated output and reducing reliance on the model's internal parametric knowledge.

1-12-4 Dense Vector Embeddings and FAISS

The retrieval step in retrieval-augmented systems typically relies on dense vector similarity, where textual passages are encoded as high-dimensional embeddings and indexed in an approximate nearest neighbor (ANN) structure such as the Facebook AI Similarity Search (FAISS) library [14]. FAISS provides multiple indexing strategies, including flat, inverted-file, and product-quantized, each offering different trade-offs between retrieval accuracy and query latency [14]. Such indexing structures enable efficient retrieval of the top-k most relevant passages for a given query from large-scale knowledge corpora.

1-12-5 Citation Grounding and Hallucination Prevention

Hallucination prevention in retrieval-augmented systems is commonly achieved by constraining the generator to ground its outputs in retrieved evidence and to associate factual claims with verifiable sources [13], [21]. This approach, often referred to as citation grounding, ensures that generated content can be traced back to supporting passages, thereby improving reliability and interpretability. By limiting generation to information present in the retrieved context, such mechanisms reduce the likelihood of unsupported or fabricated outputs.

1-13 Conclusion

This chapter established the theoretical primitives underlying Cloud IAM Sentinel by introducing core security concepts including authentication, authorization, least privilege, and defense in depth, along with cloud computing as defined by NIST and the AWS shared responsibility model that defines the customer's authorization boundary. It further detailed the IAM service, covering its principal types, credential mechanisms, policy structure, and deterministic evaluation model, as well as identity and organization-level controls that limit effective permissions. Building on this, the chapter formalized RBAC/ABAC and defined attribute mutability as a core requirement for modeling state changes based on tags. It then defined privilege escalation as a bounded reachability problem on a state-augmented permission graph, identified AWS CloudTrail as the authoritative source of dynamic evidence, and introduced the MITRE ATT&CK Cloud taxonomy, the CIS AWS Foundations Benchmark, AWS Security Hub FSBP, and NIST SP 800-53 as the primary frameworks for labeling and compliance mapping. Finally, it outlined the foundations of LLM and RAG architecture with a citation-grounding mechanism that ensures verifiable and non-hallucinated outputs.

Chapter Two

Previous Studies

2-1 Introduction

This chapter surveys recent work on AWS and cloud Identity and Access Management security analysis published between 2024 and 2026. The relevant literature is organized by approach category, including foundational IAM surveys, IAM audit and posture frameworks, graph-based privilege-escalation analysis, Large Language Model–based threat detection, and Cloud Security Posture Management. The chapter concludes with two comparative tables. positions research approaches along a set of key design dimensions, while the second compares widely used open-source IAM analysis tools based on the consolidated evaluation of Nguyen et al. [28], and the official documentation of each tool. Industry context is provided by reports from the Cloud Security Alliance [29], Microsoft [30], and the Center for Internet Security [12], all of which identify IAM misconfiguration as a dominant root cause of cloud security incidents.

2-2 Foundational IAM Surveys

Recent survey literature provides a high-level perspective on the principles and design considerations underlying cloud Identity and Access Management systems. Rather than proposing concrete analysis techniques, these works synthesize existing practices and identify recurring challenges in securing IAM configurations.

Kaleru [22] presents a 2025 survey of IAM foundations, principles, and best practices, organizing the field into six pillars: strong authentication, access controls, privilege management, lifecycle processes, intelligent monitoring, and adaptive risk management. The survey is broad and prescriptive, providing practitioner-oriented motivation for the present work.

Jain [23] presents a 2025 survey of IAM in the cloud, focused on its components, authentication mechanisms, and authorization processes. The survey reviews single sign-on, multi-factor authentication, and federated identity management as the standard building blocks of modern cloud IAM, and it highlights the central role of correct authorization in overall cloud security.

2-3 IAM Audit and Posture Frameworks

Bello et al. [24] propose in 2025 a Scalable IAM Audit Framework for Distributed Cloud Security, focusing on the horizontal scalability of IAM auditing across multi-cloud infrastructures. The framework emphasizes event aggregation and cross-account correlation. It operates at the audit-log level rather than at the policy-evaluation level, and it does not perform graph-based reachability analysis across identities or account for attribute mutability.

Bello et al. [25] complement the previous work in the same year with a Zero Trust IAM Framework for Unified Auditing in Hybrid Distributed Systems. This second framework integrates the same auditing pipeline with a zero-trust enforcement layer, extending the applicability of the audit pipeline to hybrid deployments. As in the previous work, the analysis is performed at the audit-log level and does not include privilege-graph reasoning.

2-4 Graph-Based IAM Privilege-Escalation Analysis

Hu, Khurshid, and Tiwari [19] present TAC, a 2025 greybox penetration-testing approach for IAM privilege escalation. TAC formalizes the IAM authorization surface as a Permission Flow Graph and combines a query-based abstraction with reinforcement learning and graph-neural-network optimization to reduce customer disclosure during third-party scanning. The Permission Flow Graph formalism defined by TAC is conceptually similar to privilege-graph approaches in the literature, with two key distinctions. First, TAC operates in a greybox setting in which the scanning service does not have full visibility into the IAM configuration, whereas whitebox approaches assume access to a complete inventory. Second, TAC does not model tag-mutation actions as state-transition edges, limiting its ability to capture attribute-dependent escalation paths.

Nguyen, Ho, and To [28] present SkyEye, a 2025 open-source framework for AWS IAM privilege-escalation discovery. SkyEye addresses the boundary-scope problem, namely escalation paths that require simultaneous reasoning across identity-based policies, Service Control Policies, permission boundaries, and trust relationships. It introduces a deep comparison model for policy documents to detect subtle differences between policy versions and maps identified escalation paths to MITRE ATT&CK techniques. SkyEye represents one of the closest approaches to comprehensive privilege-graph analysis. However, it does not treat tag-mutation actions as first-class edges, and its detection layer relies on rule-based mapping rather than grounded generation.

Madireddy [26] presents in 2025 a Graph Neural Network based Adaptive Threat Detection framework for cloud IAM logs. The framework models IAM audit trails as heterogeneous dynamic graphs with attention-based aggregation and continuously updated embeddings, enabling the detection of latent user–resource interaction patterns. This line of work applies graph reasoning to observed activity rather than to authorization structure, focusing on behavioral detection over logged events.

2-5 Large-Language-Model-Based IAM Threat Detection

Adediran, Awuson-David, and Ahmed [21] propose in 2026 a two-step Retrieval-Augmented Generation pipeline using Gemini 2.5 Pro for AWS CloudTrail threat detection and MITRE ATT&CK mapping. Their evaluation on a dataset of two hundred CloudTrail events reports an accuracy of 78% and an F1 score of 79%, representing a substantial improvement over the same model without retrieval augmentation. The approach demonstrates the effectiveness of combining retrieval mechanisms with language models to improve the reliability of security analysis.

This line of work highlights the potential of LLM-based detection for cloud security but remains dependent on probabilistic generation and does not incorporate deterministic grounding constraints or state-aware reasoning over IAM authorization structures.

2-6 CSPM Quality and False-Positive Reduction

Dikshant and Verma [27] present in 2025 a validation-driven methodology for reducing false positives in Cloud Security Posture Management (CSPM) systems. Their approach integrates active behavioral testing through lightweight automated probes that simulate adversarial actions

to assess the exploitability of policy violations in real time. This method aims to distinguish between configurations that are theoretically insecure and those that are practically exploitable.

This line of work highlights the importance of validating security findings against runtime behavior. However, it relies on active interaction with cloud systems and real-time execution of probes, introducing operational overhead and potential side effects. In contrast, static analysis approaches avoid such interaction by reasoning over configuration state alone, trading runtime validation for scalability and safety.

2-7 Comparative Analysis of Research Papers

Table 2-1 compares the seven practical research papers reviewed in Sections 2-3 through 2-6 across a set of six design dimensions. These dimensions include the approach category adopted by each system, whether IAM Condition operators are evaluated during policy analysis, whether attribute mutability is modeled through tag-mutation actions as state transitions, whether dynamic CloudTrail evidence is incorporated, whether Large Language Models with citation grounding are used for alert generation, and whether alerts are mapped to MITRE ATT&CK technique identifiers.

Table 2-1: Comparison of Research Papers Against the Proposed Framework

System / Reference	Approach	IAM Conditions	Tag Mutation	CloudTrail	Grounded LLM	MITRE ATT&CK
Bello et al. [24]	Audit-log analysis	No	No	Indirect	No	No
Bello et al. [25]	Audit + zero-trust enforcement	No	No	Indirect	No	No
Hu et al. (TAC) [19]	Symbolic graph + GNN	Partial	No	No	No	No
Nguyen et al. (SkyEye) [28]	Symbolic graph + policy diff	Yes	No	Partial	No	Yes
Madireddy (GNN-IAM) [26]	Graph Neural Network	No	No	Yes	No	No
Adediran et al. (RAG-AWS) [21]	RAG-LLM	No	No	Yes	Yes	Yes
Dikshant and Verma [27]	Active validation	Indirect	No	No	No	No
Cloud IAM Sentinel	Hybrid (rules + graph + RAG-LLM)	Yes	Yes	Yes	Yes	Yes

The table highlights a residual gap in the current literature. No surveyed system simultaneously evaluates IAM Condition operators (as in symbolic graph approaches such as TAC and SkyEye), models tag-mutation actions as state transitions (absent in all surveyed systems), incorporates

dynamic CloudTrail evidence (as in audit-log and graph neural network approaches), and produces alerts in which every claim is grounded in authoritative sources (as in retrieval-augmented approaches such as Adediran et al.). The proposed approach integrates all four properties within a unified and deterministic analysis pipeline.

2-8 Comparative Analysis of Practical Tools

The literature surveyed in the previous sections is dominated by academic prototypes. This section shifts focus to open-source tools currently used in practice for AWS IAM analysis. The comparison draws on the consolidated evaluation presented by Nguyen et al. [28] as well as the official documentation of each tool. The tools considered include Principal Mapper (PMapper), PACU, CloudFox, ScoutSuite, Cloudsplaining, Prowler, and SkyEye, together with the proposed approach.

Table 2-2 summarizes the capabilities of each tool across seven design dimensions: static rule-based analysis of IAM configurations, construction of privilege graphs and reachability analysis, evaluation of IAM Condition operators, modeling of tag-mutation actions as state transitions, correlation with dynamic CloudTrail events, use of Large Language Models with citation grounding, and mapping of alerts to MITRE ATT&CK technique identifiers.

Table 2-2: Comparison of Practical IAM Analysis Tools Against the Proposed Framework

Tool	Static Rules	Graph Reachability	IAM Conditions	Tag Mutation	CloudTrail Correlation	Grounded LLM	MITRE ATT&CK
PMapper (Principal Mapper)	No	Yes	No	No	No	No	No
PACU	No	No	No	No	Indirect	No	No
CloudFox	Partial	No	No	No	No	No	No
ScoutSuite	Yes	No	No	No	No	No	No
Cloudsplaining	Yes	No	No	No	No	No	No
Prowler	Yes	No	No	No	No	No	No
SkyEye	Yes	Yes	Yes	No	Partial	No	Yes
Cloud IAM Sentinel	Yes	Yes	Yes	Yes	Yes	Yes	Yes

The table demonstrates that no existing open-source tool simultaneously supports static posture analysis, graph-based reachability with IAM Condition evaluation, modeling of attribute mutability, dynamic CloudTrail correlation, and citation-grounded Large-Language-Model reasoning within a single pipeline. The proposed approach occupies this position in the design space, providing an integrated capability that is absent from all surveyed tools.

2-9 Discussion and Justification of the Proposed Framework

Three observations from the 2024-2026 literature reviewed in this chapter justify the design of Cloud IAM Sentinel. First, IAM misconfiguration and the resulting privilege escalation remain a dominant root cause of cloud-security incidents, as documented by the Cloud Security Alliance [29], by Microsoft [30], and by Bello et al. [24]. Second, prior work has converged on graph reachability as the primary formalism for static privilege-escalation analysis (Hu et al. [19]; Nguyen et al. [28]) and on graph-based representations for dynamic IAM-log analysis (Madireddy [26]), however, no surveyed system models attribute mutability, that is, the property by which modifications to tag attributes can alter subsequent authorization outcomes. Third, retrieval-augmented architectures have demonstrated improved accuracy in Large Language Model–based threat detection (Adediran et al. [21]), but existing approaches do not enforce deterministic grounding constraints on generated outputs.

The proposed approach synthesizes these observations into a unified design. The framework constructs a deterministic privilege graph that evaluates IAM Condition operators, augments the graph with tag-mutation edges representing attribute-dependent state transitions, and incorporates CloudTrail-based correlation with alert generation supported by citation-grounded retrieval mechanisms. No system surveyed in the 2024–2026 literature combines all of these properties within a single pipeline, and this integration constitutes the primary contribution of the work.

2-10 Conclusion

This chapter surveyed recent literature (2024–2026) on AWS and cloud IAM security analysis across five relevant approach categories, presented two comparative tables evaluating existing systems and tools, and identified a residual gap in the current state of the art. Specifically, no surveyed approach simultaneously combines IAM Condition evaluation, attribute-mutability modeling, dynamic CloudTrail correlation, and citation-grounded Large Language Model reasoning. The next chapter presents the architecture and design of the proposed framework.

Chapter Three
Proposed Solution

3-1 Introduction

This chapter presents the proposed solution, Cloud IAM Sentinel, and describes its overall architecture, the modules that comprise it, and the way these modules interact. The chapter begins with the high-level architecture diagram that summarizes the data flow from the external AWS evaluation tenant through to the final correlated alerts, then proceeds to describe each layer in turn: the data collection layer, which ingests IAM configuration and CloudTrail events into a unified inventory, the static analysis phases, which evaluate posture rules and construct a privilege-escalation graph, and the dynamic correlation phase, which maps observed activity to MITRE ATT&CK techniques and produces grounded alerts through a Retrieval-Augmented Generation pipeline. Each phase is described in terms of its inputs, processing logic, and outputs, with reference to the relevant theoretical foundations and design considerations.

3-2 Framework Architecture Overview

The proposed framework occupies an intermediary position within the AWS IAM security assessment workflow, operating between the AWS account under analysis, and the security engineer who consumes the assessment results. Figure 7 shows this position. The framework ingests IAM configuration data, CloudTrail event logs, and the Amazon Elastic Compute Cloud instance inventory from the cloud account, and draws on external authoritative sources including MITRE ATT&CK, CIS AWS Foundations Benchmark, AWS official documentation, and NIST SP 800-53 Revision 5, to ground its analysis. It produces posture findings, privilege-escalation paths, citation-grounded alerts, and remediation guidance, which are delivered to the security engineer in JavaScript Object Notation and Comma-Separated Value formats.

The framework performs no write operations on the cloud account. Its read-only data ingestion model ensures safe deployment as an assessment tool, without introducing any risk of unintended modifications to the target environment.

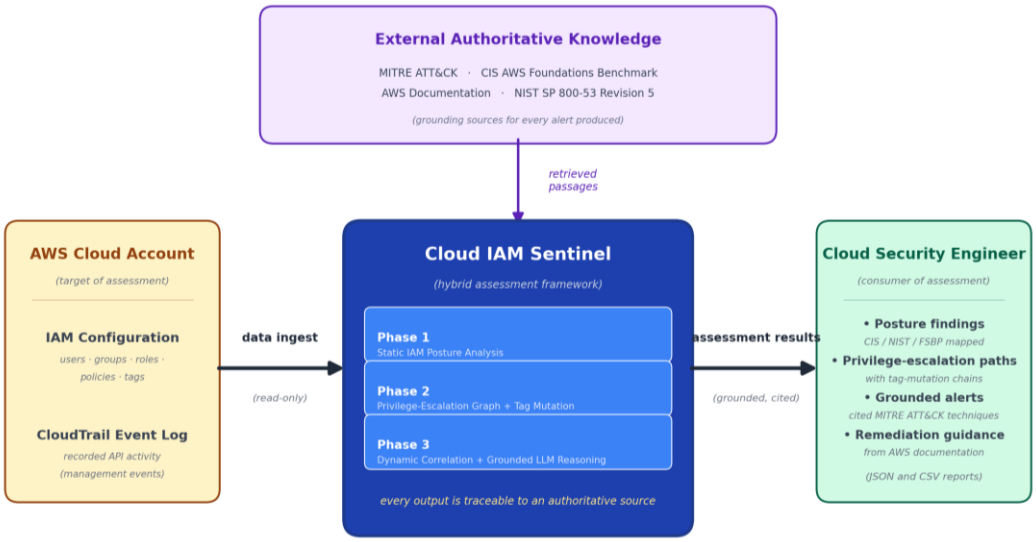

Figure 7: Cloud IAM Sentinel as an Assessment Intermediary in the AWS IAM Security-Assessment Workflow

The proposed framework is organized into three sequential analysis phases that operate over a shared data foundation. Figure 8 shows the end-to-end data flow from the external AWS account under analysis to the final correlated alerts.

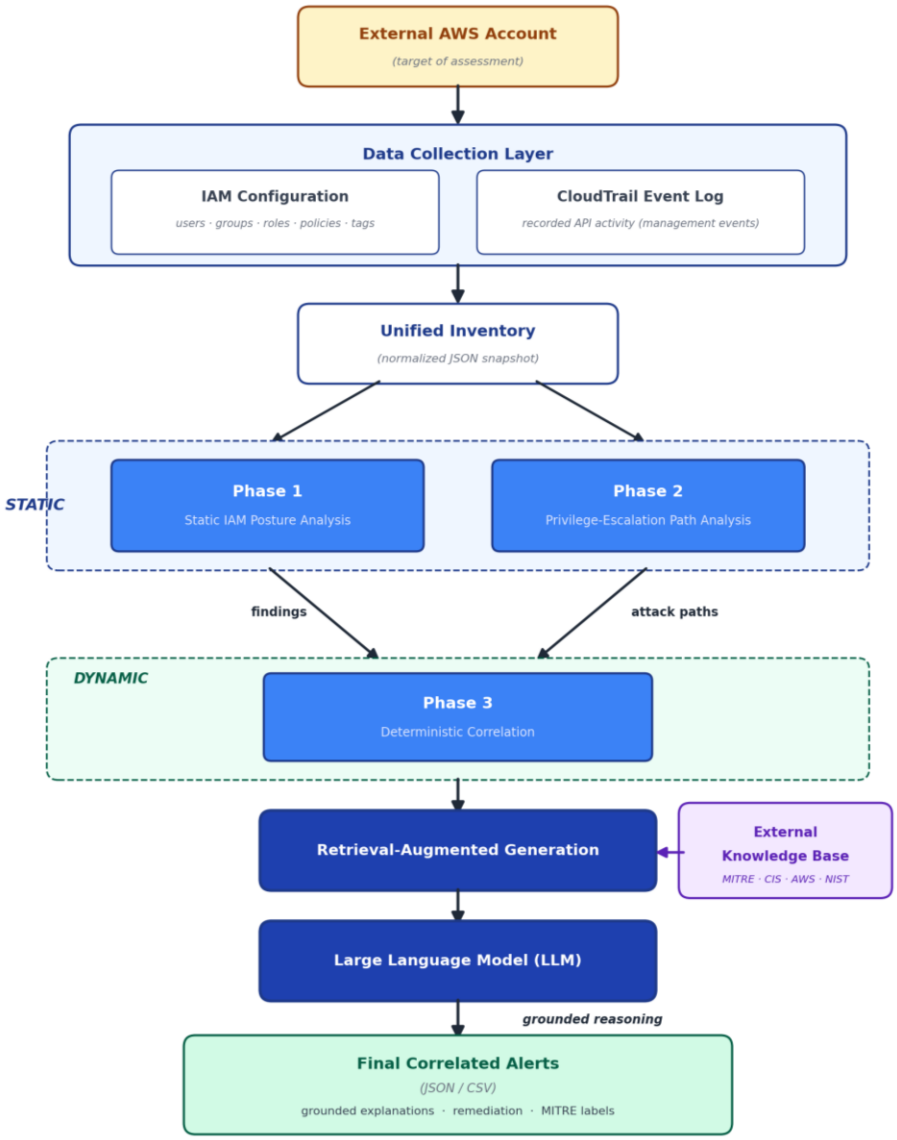

Figure 8: Cloud IAM Sentinel — Framework Architecture

The data collection layer ingests two complementary sources of evidence from the external AWS account: the IAM configuration, which consists of users, groups, roles, policies, trust relationships, and tags, and the CloudTrail event log, which provides an immutable record of Application Programming Interface activity recorded by the AWS platform [10]. These inputs are normalized into a unified inventory that is consumed by all subsequent phases.

The static portion of the pipeline consists of Phase One, which evaluates posture rules over the IAM configuration to produce findings, and Phase Two, which constructs a privilege-escalation

graph over the same inventory to identify reachable attack paths. The dynamic portion comprises Phase Three, which correlates the static outputs with the CloudTrail evidence and processes the result through a Retrieval-Augmented Generation pipeline supported by an external knowledge base of MITRE ATT&CK Cloud techniques [11], AWS service authorization references [1], and CIS AWS Foundations Benchmark control descriptions [12].

A Large Language Model generates grounded reasoning for each alert, and the output of the model is filtered by a deterministic citation guard before being emitted as a set of correlated alerts in JavaScript Object Notation and Comma-Separated Value formats. Each alert includes a grounded explanation, recommended remediation, and corresponding MITRE ATT&CK technique labels.

3-3 Data Collection Layer

The data collection layer is responsible for extracting and structuring data from the external AWS account into inputs required by the analysis phases. It collects two complementary classes of data: the static IAM configuration and the dynamic CloudTrail event stream.

3-3-1 IAM Configuration Collection

The IAM configuration collector queries the AWS IAM service using read-only Application Programming Interface operations defined in the project permission policy [1]. The collector retrieves the complete set of users, groups, roles, managed policies, inline policies, group memberships, role-attached policies, trust policies, attached tags, password policies, and Multi-Factor Authentication device assignments. The collected data are normalized into a JavaScript Object Notation inventory. All analysis phases operate on this pre-collected and normalized snapshot of the account state, without requiring any direct or live interaction with the AWS environment during execution. The collector additionally retrieves the Amazon Elastic Compute Cloud instance inventory, which is required by Phase Two to model PassRole-via-EC2 escalation paths.

3-3-2 CloudTrail Event Collection

The CloudTrail collector retrieves management events using the cloudtrail:LookupEvents Application Programming Interface operation for a configurable time window [10]. Each retrieved event is preserved in its native JavaScript Object Notation format to ensure that no fields are altered prior to deterministic processing in the dynamic analysis phase. The collector can alternatively process CloudTrail data from a pre-existing JavaScript Object Notation file, allowing analysis to be performed on previously collected event records.

3-3-3 Unified Inventory

The unified inventory is a data structure that integrates the IAM configuration, the Amazon Elastic Compute Cloud inventory, and the CloudTrail events into a single in-memory representation. The unified inventory is the only data exchange interface between the data collection layer and the three analysis phases. The inventory is keyed by Amazon Resource Name where applicable, enabling efficient cross-references between policy statements, identities, and recorded events without requiring multiple parsing passes.

3-4 Phase One — Static IAM Analysis

Phase One evaluates a catalog of static posture rules over the unified inventory and produces a set of findings. It represents the static counterpart of a Cloud Security Posture Management approach, with two distinguishing properties: each rule is grounded in an authoritative compliance baseline, and each finding explicitly identifies the policy statement, principal, or resource that triggered it.

3-4-1 Rule Catalog Architecture

The rule catalog is partitioned into two sub-catalogs: a Role-Based Access Control sub-catalog that contains thirty-nine rules and an Attribute-Based Access Control sub-catalog that contains six rules. Each rule is implemented as a Python class that exposes a single evaluate method which takes the unified inventory as input and returns either a Finding object or no result when the rule does not apply. Each rule is accompanied by a metadata file with five required fields: the severity assigned to the finding, the basis for that severity, the authoritative source from which the rule is derived, the identifier of the corresponding CIS Critical Security Control [12], and the identifier of the corresponding NIST SP 800-53 Revision 5 control [15].

3-4-2 RBAC Posture Rules

The Role-Based Access Control sub-catalog addresses categories of misconfiguration that have historically contributed to major cloud-security incidents. Representative categories include the use of long-lived access keys, the absence of Multi-Factor Authentication for the root user and on console-enabled IAM users, overly permissive administrator-level managed policies, weak password policies, unused credentials, and direct policy attachments to users in violation of the recommended group-based assignment model. Each rule in this sub-catalog cites its authoritative source and corresponding CIS and NIST control identifiers, so that every finding is auditable.

3-4-3 ABAC Posture Rules

The Attribute-Based Access Control sub-catalog addresses misconfiguration patterns that arise specifically from attribute-based authorization. Representative categories include the use of fail-open Condition operators such as StringEqualsIfExists and Null without a corresponding fail-closed constraints, the absence of separation-of-duty controls when multiple principals share the same tag, and policies that grant the iam:TagUser action over a principal whose attributes are referenced by a downstream aws:PrincipalTag condition [4].

3-4-4 Output Schema

Each finding produced by Phase One is serialized into a JavaScript Object Notation document and a corresponding Comma-Separated Value record. The JavaScript Object Notation record includes the rule identifier, severity level, authoritative source, control identifier, principal or resource identifier, the policy statement that triggered the finding, and a human-readable description. A schema validator enforces the presence of all required fields before a finding is emitted, ensuring that no malformed output is propagated to subsequent phases.

3-5 Phase Two — Privilege-Escalation Path Analysis

Phase Two constructs a directed privilege-escalation graph over the unified inventory and identifies the set of reachable paths from each principal to privileged target. The phase is the principal innovation of the framework, since it explicitly evaluates IAM Condition operators and models tag-mutation actions as state transitions.

3-5-1 Graph Construction

The graph builder iterates over the unified inventory and emits one node per principal (user, group, role, federated identity, service principal) and one node per resource that participates in a privilege-granting relationship (Amazon Elastic Compute Cloud instance role, Amazon Simple Storage Service bucket, AWS Key Management Service key, AWS Lambda function). Edges are generated in two passes. The first pass enumerates the privilege-granting actions present in the inventory, including iam:PassRole, sts:AssumeRole, iam:CreateAccessKey, iam:UpdateAssumeRolePolicy, iam:AttachRolePolicy, iam:PutRolePolicy, and tag-mutation actions. The second pass evaluates each candidate edge according to AWS IAM policy semantics, including application of the deny-overrides evaluation model and resolution of all applicable Condition operators [1].

3-5-2 IAM Condition Evaluation

The Condition evaluator implements the operators most frequently encountered in production policies: StringEquals, StringNotEquals, StringEqualsIfExists, StringLike, Bool, ArnLike, ArnEquals, Null, ForAllValues:StringEquals, and ForAnyValue:StringEquals [4]. For each Condition, the evaluator returns one of three verdicts: true, false, or unknown. An unknown verdict results in the candidate edge being marked as conditional rather than admitted unconditionally, preserving soundness in the presence of operators that are not yet implemented.

3-5-3 Tag-Mutation Edges and Joint-State Reachability

Tag-mutation actions are modeled as a distinct edge class. When the graph engine encounters a principal that has permissions such as iam:TagUser, iam:UntagUser, iam:TagRole, iam:UntagRole, ec2:CreateTags, or s3:PutBucketTagging permission, it emits an edge that mutates the tag state of the principal or the target resource. The reachability search is performed over the joint space of node identity and tag-state hash, allowing the same node to be visited multiple times under different attribute configurations and enabling representation of escalation paths that requires modifying a tag before traversing a downstream edge. This design directly realizes the formal notion of attribute mutability defined by NIST Special Publication 800-205.

3-5-4 Bounded Breadth-First Search

Reachability is computed by bounded breadth-first search from each principal to each privileged target. The search depth is limited by a configurable parameter, with a default value of three, ensuring tractability on production-scale inventories while preserving coverage of known escalation paths documented in the IAM Vulnerable benchmark [6] and the Pathfinding.cloud catalog [7]. Each discovered path is annotated with the sequence of edges traversed, the authoritative-source associated with each edge type, and the Condition verdicts encountered along the path.

3-5-5 Service Control Policy and Permission Boundary Filtering

After a candidate path is enumerated, it is post-filtered using Service Control Policy and Permission Boundary constraints. The Service Control Policy filter enforces organization-level guardrails, while the Permission Boundary filter applies the maximum permission set defined for each principal. If a path is denied by either filter, it is annotated as blocked by a guardrail or permission boundary rather than reported as a successful escalation. This ensures that the analysis remains consistent with AWS IAM evaluation semantics.

3-5-6 Output Schema

Each discovered path is serialized into a JavaScript Object Notation document and a corresponding Comma-Separated Value record. The JavaScript Object Notation record includes the source principal, the target privilege, the sequence of traversed edge types, Service Control Policy and Permission Boundary annotations, and the citation linking each action in the path to the AWS IAM action reference [1].

3-6 Phase Three — Deterministic Correlation and Grounded LLM Reasoning

Phase Three correlates the outputs of Phase One and Phase Two with the dynamic CloudTrail evidence to produce the final alerts delivered to the responder. The phase combines a deterministic mapping layer with a Retrieval-Augmented Generation layer, the latter constrained by a citation-grounding guard that prevents hallucination content.

3-6-1 CloudTrail Event Ingestion

The CloudTrail ingestion module processes events collected in the data collection layer, normalizing the userIdentity field across the AssumedRole, IAMUser, AWSService, and Root identity types. Events are grouped into time-windowed sessions per principal, which serve as the unit of correlation in subsequent processing.

3-6-2 Deterministic MITRE ATT&CK Mapping

The MITRE mapper applies a deterministic lookup table that maps each CloudTrail eventName to a MITRE ATT&CK technique identifier [11]. The mapping is curated from the MITRE ATT&CK Cloud (IaaS) matrix and includes techniques such as Valid Accounts: Cloud Accounts (T1078.004), Account Manipulation: Additional Cloud Credentials (T1098.001), Account Manipulation: Additional Cloud Roles (T1098.003), Create Account: Cloud Account (T1136.003), and Impair Defenses: Disable or Modify Cloud Logs (T1562.008). The deterministic nature of the mapping ensures that each alert is associated with a verifiable technique identifier.

3-6-3 Correlation with Static Outputs

For each session, the correlator matches the session’s principal identifier against records produced in Phase One and Phase Two. A correlated alert is generated when a session contains an event whose mapped technique corresponds to a privilege-granting action identified either in the Phase One rule catalog or in a Phase Two attack path. This correlation step transforms an isolated CloudTrail event into an observation grounded in a potential privilege-escalation path.

3-6-4 Retrieval-Augmented Generation Layer

For each correlated alert, the Retrieval-Augmented Generation layer constructs a query composed of the CloudTrail evidence, the corresponding Phase One finding or Phase Two path, and the associated MITRE ATT&CK technique identifier. The query is processed by a dense retriever that searches the external knowledge base described in the next subsection and returns the top-k most relevant passages. The passages are combined with the evidence and technique label and passed to the Large Language Model adapter to generate a grounded explanation.

3-6-5 External Knowledge Base

The external knowledge base is a curated collection of authoritative documents that includes the MITRE ATT&CK technique pages [11], AWS service authorization references [1], the AWS IAM Action Reference [1], the CIS AWS Foundations Benchmark control descriptions [12], and AWS official remediation playbooks. The knowledge base is indexed by the Facebook AI Similarity Search library [14] over dense vector embeddings and is rebuilt by a dedicated utility whenever any source is updated.

3-6-6 LLM Adapter and Backend Selection

The Large Language Model adapter exposes a single interface that supports two backends. The first is a deterministic null backend that performs no generation and serves as the no-LLM experimental control: every Large-Language-Model field in the resulting alert is left empty, so the alert contains only the deterministic correlation tier, the MITRE ATT&CK technique mapping, the evidence identifiers, and the retrieved knowledge passages. The second is an OpenAI-compatible backend that talks to any inference endpoint conforming to the OpenAI Application Programming Interface; the default runtime path uses this backend to serve a general-purpose open-weight Large Language Model (Llama 3.1-8B-Instruct, in the Q4_K_M quantization) through a local Ollama inference daemon at the loopback address. Every alert records the backend that produced it through a dedicated metadata field, which permits direct experimental comparison of the no-LLM baseline against the general-purpose model under identical evidence and prompt conditions. The substitution of a cybersecurity-specialized Large Language Model in place of the general-purpose model, and the comparison of the two specializations under the same evidence and prompt conditions, is reserved for future work.

3-6-7 Hallucination Guard

The hallucination guard is a deterministic post-processing module that inspects every Large Language Model output and verifies that all factual claims are supported by the retrieved passages or the input evidence. Any output that introduces a MITRE technique identifier, a CIS control number, an AWS service name, or a NIST control identifier not present in the retrieved passages or CloudTrail evidence is rejected. Rejected outputs are replaced with a deterministic alert containing only the evidence and the deterministic MITRE mapping. This mechanism ensures that no alert delivered to the responder contains unsupported claims.

3-6-8 Final Correlated Alerts

Alerts are serialized into JavaScript Object Notation document and a corresponding Comma-Separated Value record. Each alert includes the session principal, the associated CloudTrail events, MITRE ATT&CK technique labels, the matched Phase One finding identifier or Phase

Two path identifier, , the grounded explanation generated by the Large Language Model, the cited knowledge base passages, recommended remediation, and the backend that produced the alert.

3-7 Inter-Phase Data Flow and Output Formats

The framework is implemented as a unified Python pipeline that shares a common AWS inventory between Phase One and Phase Two through a single runner entry point, and that processes the resulting findings and attack paths in Phase Three through a dedicated Phase Three runner. Both execution paths emit their outputs as JavaScript Object Notation documents validated against phase-specific schemas, alongside corresponding Comma-Separated Value files for analyst-facing review. The pipeline operates deterministically in configurations without Large Language Model generation, and remains deterministic up to the configured temperature parameter in LLM-augmented modes.

3-8 Conclusion

This chapter described the proposed Cloud IAM Sentinel framework, including its three-phase architecture, the data collection layer that produces the unified inventory, the static analysis layer that evaluates posture rules and computes privilege-escalation reachability with explicit IAM Condition evaluation and tag-mutation modeling, and the dynamic correlation layer that integrates CloudTrail evidence with grounded Large Language Model reasoning. It outlined the modules within each phase, their inputs and outputs, and the schemas that govern data exchange. The next chapter presents the implementation environment and tools used to realize the proposed framework.

Chapter Four
Practical Implementation

4-1 Introduction

This chapter presents the practical implementation of Cloud IAM Sentinel and the evaluation environment in which the framework operates. It begins by describing the programming environment and the read-only AWS permission policy governs all interactions with the cloud account under analysis. The chapter then enumerates the Phase One rule catalogue, organized by rule family, and justifies each family with respect to whether equivalent protection could be provided by network-layer controls or AWS-native services. The implementation of Phase Two and Phase Three is subsequently described. The chapter concludes with a description of the open ground-truth datasets used to evaluate the framework and the open-source comparison tools deployed alongside Cloud IAM Sentinel for the head-to-head evaluation presented in Chapter Five.

4-2 Implementation Environment

4-2-1 Programming Language and Libraries

The framework is implemented in Python (version 3.10 or 3.11). The data collection layer relies on the official AWS Software Development Kit for Python (boto3), which provides the read-only Application Programming Interface clients used to retrieve IAM configuration data, Amazon Elastic Compute Cloud inventory, and CloudTrail events. The Phase Two privilege-graph engine is built on the NetworkX library for in-memory directed-graph construction and traversal. The Phase Three retrieval layer indexes the external knowledge base using the Facebook AI Similarity Search library [14] over dense vector embeddings produced by an open-weight sentence-embedding model and the Large Language Model is served locally through a self-hosted inference daemon exposing an OpenAI-compatible Application Programming Interface on the loopback interface.

4-2-2 Read-Only AWS Permission Policy

The framework operates exclusively under a least-privilege, read-only Application Programming Interface policy attached to the user or role executing the assessment. The policy grants IAM read operations required to enumerate users, groups, roles, managed and inline policies, password policies, Multi-Factor Authentication device assignments, and the credential report. It also includes the CloudTrail read operation required to retrieve recent management events during execution. and an AWS Security Token Service operation used to identify the executing principal for the operator-trail-exclusion logic in Phase Three.

No write operations are permitted, and no identities are created, modified, or deleted by the framework. Cloud IAM Sentinel therefore satisfies the principle of least privilege at the deployment level while enforcing it at the analysis level.

4-3 Phase One — Static IAM Rule Catalog

4-3-1 Rule Organization into Detection Families

The Phase One catalog contains forty-five rules organized into six detection families. Thirty-nine rules address the Role-Based Access Control surface and are summarized at the family level in

Table 4-1. Six rules address the Attribute-Based Access Control surface and are presented at the rule level in Table 4-2.

Each rule is implemented as a Python class exposing a single evaluate method and is accompanied by a metadata file that specifies the rule's severity, the basis for that severity, the authoritative source from which the rule is derived, the corresponding CIS Critical Security Control identifier [12], and the corresponding NIST Special Publication 800-53 Revision 5 control identifier [15].

Table 4-1: Role-Based Access Control Rules

Family	Rules	Detection focus	Authoritative basis
A — Identity policy structure	19	Admin-equivalent managed policy attachments to non-administrative principals; privilege-escalation primitives across identity, group, role, inline, and trust policies; full-access grants on sensitive services such as CloudTrail and AWS Key Management Service; overly permissive trust-policy principals; and missing aws:SourceArn or aws:SourceAccount conditions in cross-service trust relationships (the confused-deputy pattern documented by AWS).	CIS AWS Foundations Benchmark v3.0.0 §1.16, §1.17; NIST SP 800-53 Rev. 5 AC-6 (Least Privilege), AC-3 (Access Enforcement)
C — Credential and authentication state	8	Presence of access key on the AWS account root user; absence of Multi-Factor Authentication for the root user in both software and hardware forms; absence of Multi-Factor Authentication for console-enabled IAM users; centralized management of root credentials at the AWS Organizations level; multiple active access keys associated with a single user; and recent root-account activity as reported in the credential report.	CIS AWS Foundations Benchmark v3.0.0 §1.4–§1.10; NIST SP 800-53 Rev. 5 IA-2 (Identification and Authentication), AC-2 (Account Management)
D — Credential lifecycle	4	Access keys not rotated within ninety days; access keys and console access unused for forty-five days, increasing the attack surface; and expired Transport Layer Security server certificates retained in IAM that may be inadvertently reused.	CIS AWS Foundations Benchmark v3.0.0 §1.14, §1.12, §1.13; NIST SP 800-53 Rev. 5 AC-2(3) (Disable Inactive Accounts), IA-5 (Authenticator Management)
E — Account-level password policy	6	Minimum password length of fourteen characters; password reuse-prevention setting of twenty-four; and enforcement of four character classes (lowercase, uppercase, numeric, and symbolic). The length and reuse rules are classified as medium severity according to CIS, while the character-class requirements are classified as low severity, as password length is the dominant factor in entropy.	CIS AWS Foundations Benchmark v3.0.0 §1.8, §1.9; NIST SP 800-53 Rev. 5 IA-5(1) (Password-Based Authentication)
F — Provisioning hygiene	2	Absence of dedicated security-audit and incident-response roles separate from administrative	CIS AWS Foundations Benchmark v3.0.0 §1.17;

		principals. These rules enforce governance by ensuring that audit and incident-response activities are performed using purpose-built roles rather than shared high-privilege credentials.	NIST SP 800-53 Rev. 5 AC-5 (Separation of Duties), AC-6(2) (Non-Privileged Access for Non-Security Functions)

Table 4-2: Attribute-Based Access Control Rules (Family B)

Rule	Pattern detected	Required condition key	Authoritative basis
PassRole without pass condition	An iam:PassRole grant that omits the iam:PassedToService condition, allowing the holder to attach an IAM role to arbitrary AWS services rather than a specific intended service.	iam:PassedToService	AWS IAM User Guide — Granting a user permissions to pass a role to an AWS service [1]; NIST SP 800-162 §3.2 (subject attribute scoping)
Tag-write without aws:TagKeys	A tag-write action (e.g., iam:TagUser, iam:TagRole, ec2:CreateTags, s3:PutBucketTagging) granted without an aws:TagKeys allowlist, permitting unrestricted modification of tag keys.	aws:TagKeys	AWS IAM Tutorial — ABAC condition operators [9]; NIST SP 800-162 §3.4 (attribute write controls)
Sensitive IAM action without MFA condition	A sensitive IAM action (e.g., iam:CreateAccessKey, iam:UpdateAssumeRolePolicy, iam:AttachRolePolicy) granted without an aws:MultiFactorAuthPresent condition, allowing execution without multi-factor authentication.	aws:MultiFactorAuthPresent	NIST SP 800-162 §3.1.3.5 (environment condition / authentication assurance); CIS AWS Foundations Benchmark v3.0.0 §1.10
Delegated creation without permission boundary	A delegated identity-creation grant (e.g., iam:CreateUser, iam:CreateRole) without an iam:PermissionsBoundary condition, allowing creation of principals that are not constrained by the delegating entity's permission boundary.	iam:PermissionsBoundary	AWS IAM User Guide — Permissions boundaries for IAM entities [1]; NIST SP 800-162 §3.6 (delegation containment)
Resource creation without aws:RequestTag	A resource-creation action (e.g., ec2:RunInstances, s3:CreateBucket) granted without enforcement of aws:RequestTag, allowing creation of untagged resources that bypass subsequent ABAC policies based on aws:ResourceTag.	aws:RequestTag	AWS IAM Tutorial — ABAC for AWS resources [9]; NIST SP 800-162 §3.4 (attribute write controls)

Wildcard policy without tag conditions	An identity policy that grants wildcard actions over wildcard resource in an environment managed under Attribute-Based Access Control, but lacks aws:PrincipalTag or aws:ResourceTag conditions to constrain the grant.	aws:PrincipalTag, aws:ResourceTag	NIST SP 800-162 §3.5 (attribute-based decision); AWS IAM Condition Operators reference [4]

4-3-2 Family A — Identity Policy Structure

The nineteen rules of Family A operate over the inventory of users, groups, and roles. Each rule systematically evaluates all attached managed and inline policies associated with each principal. The corresponding policy documents are retrieved from the cached inventory, with a fallback to the IAM policy collection by Amazon Resource Name when necessary. For each policy, the evaluation iterates through every Statement element and applies a set of predefined pattern-based checks.

A shared utility module provides reusable pattern primitives that encapsulate common detection logic. These primitives are used by all rules in the family to ensure consistent evaluation. They include checks for administrative-level permissions (such as wildcard actions over wildcard resources), detection of specific high-risk or privileged actions, identification of unbounded resource scopes, and recognition of well-known high-privilege managed policies. By centralizing this logic, the framework avoids duplication and ensures that all rules apply the same criteria in a uniform manner.

Each rule evaluates whether any policy statement matches a risky pattern. When a match is found, the rule generates a violation record that captures detailed context, including the principal type, principal identifier, policy type (inline or attached), policy name, the index of the statement within the policy, and a human-readable explanation of the issue. If no matches are found, the rule returns a PASS verdict. If one or more violations are detected, the rule returns a FAIL verdict along with the list of corresponding violation records.

Representative rules in this family include detection of administrator-level access granted without Multi-Factor Authentication, identification of privilege-escalation patterns based on curated sets of sensitive actions, detection of cross-service trust policies that lack aws:SourceArn or aws:SourceAccount conditions, and identification of policies that are directly attached to users instead of being assigned through groups or roles.

4-3-3 Family B — Attribute-Based Access Control

A shared utility module provides a set of helper functions that check whether specific condition keys are present inside the Condition element of a policy statement. Each helper function corresponds to one required ABAC constraint, such as verifying the presence of iam:PassedToService, aws:TagKeys, or aws:MultiFactorAuthPresent.

During evaluation, each rule applies these checks to determine whether a policy statement includes the necessary restrictions. If a required condition key is present, the permission is considered

constrained. If the condition key is missing, the permission is treated as unsafe because it allows broader or unintended access than intended.

When such a missing condition is detected, the rule generates a structured violation record. This record includes the policy name, the index of the statement in which the issue was found, and the specific condition key that is absent. These structured records are later used in Phase Three to correlate static policy weaknesses with CloudTrail events that trigger the same permission patterns.

4-3-4 Family C — Credential and Authentication State

The eight rules of Family C operate on credential-state data derived from the credential report and from IAM Multi-Factor Authentication device listings, both of which are collected and stored in the unified inventory. These rules evaluate the configuration and usage state of credentials for both the root account and IAM users.

The analysis includes detection of active access keys on the root account, absence of Multi-Factor Authentication for the root account in both software and hardware forms, and absence of Multi-Factor Authentication for console-enabled IAM users. It also identifies users with multiple active access keys, recent usage of the root account within a defined threshold window, and IAM users provisioned simultaneously with console and programmatic credentials.

Additional checks evaluate organization-level controls, such as the root-credentials-management feature flag exposed through AWS Organizations. Each rule produces structured violation records identifying the affected principal and the relevant credential-state attributes. For rules related to the root account, the synthetic root identifier is used. A rule returns a FAIL verdict when any violation is detected and PASS otherwise.

4-3-5 Family D — Credential Lifecycle

The four rules of Family D evaluate time-based credential hygiene by analyzing timestamp fields from the credential report and the IAM server-certificate inventory. These timestamps are parsed in ISO-8601 format and compared against the current time to determine credential age and inactivity.

The rules collectively detect access keys that exceed rotation thresholds, access keys and console credentials that have remained unused beyond defined inactivity windows, and expired Transport Layer Security server certificates retained in IAM. Missing usage timestamps are interpreted as indicating credentials that have never been used.

Threshold values are configurable through rule metadata, with default values of ninety days for key rotation and forty-five days for inactivity, consistent with the CIS AWS Foundations Benchmark version three controls.

4-3-6 Family E — Account-Level Password Policy

The six rules of Family E evaluate the account-level password policy retrieved through the IAM GetAccountPasswordPolicy operation. These rules assess both quantitative requirements, such as minimum password length and password reuse prevention, and qualitative requirements related to character composition.

The evaluation verifies enforcement of lowercase, uppercase, numeric, and symbolic character requirements, and compares policy thresholds against values prescribed by the CIS AWS Foundations Benchmark. In addition, the absence of an account-level password policy is treated as a violation, as it indicates that no password constraints have been configured for IAM users.

4-3-7 Family F — Provisioning Hygiene

The two rules of Family F evaluate the presence of dedicated security-audit and incident-support roles within the AWS account. These rules inspect the inventory of IAM roles to determine whether the AWS-managed SecurityAudit and AWSSupportAccess policies are attached to any role. A violation is reported when no such role exists.

Both rules serve as governance checks rather than detection of directly exploitable misconfigurations. Their severity is therefore calibrated at low, as the absence of these roles reflects a deficiency in audit and incident-response preparedness rather than an immediately exploitable attack path.

4-3-8 Limitations of Network-Layer Controls in Substituting Phase One Rules

Network-layer security controls cannot substitute for the rules implemented in Phase One. None of the forty-five rules can be replaced by a firewall, a Network Access Control List, an AWS Security Group, the AWS Web Application Firewall, or a network-based Intrusion Detection or Intrusion Prevention System, because each rule evaluates properties of the AWS Identity and Access Management control plane, which is not observable by devices operating at OSI layers three or four.

An AWS Application Programming Interface call issued by any AWS Software Development Kit is signed by the calling principal and transmitted over Transport Layer Security to the AWS service endpoint. A firewall observes only the five-tuple of source address, destination address, protocol, and port, together with encrypted payload that terminates at the AWS endpoint. As a result, it cannot extract the API operation name, the role being passed, the tag being modified, the policy document being attached, or the Multi-Factor Authentication state of the session. A Network Access Control List operates on the same five-tuple with even less context, as it is stateless. AWS Security Groups similarly evaluate layer three and layer four metadata at the instance boundary and do not inspect the contents of AWS Application Programming Interface requests. The AWS Web Application Firewall inspects only Hypertext Transfer Protocol traffic routed through CloudFront, an Application Load Balancer, the Application Programming Interface Gateway, or AppSync; AWS service endpoints such as iam.amazonaws.com and sts.amazonaws.com are not protected by this mechanism. Network-based Intrusion Detection and Prevention systems rely on signature matching over unencrypted or decrypted traffic; a system capable of interpreting AWS IAM policy semantics would no longer function as a conventional intrusion detection system, but rather as a specialized IAM analysis engine.

The underlying reason for this separation is that an attacker in possession of valid IAM credentials, obtained through credential exfiltration, instance metadata abuse, or other means, can invoke the AWS Application Programming Interface using correctly signed requests from any network location permitted by policy. From the perspective of a network-layer control, such requests are indistinguishable from legitimate administrative activity. Authorization decisions are made within

AWS by the IAM service, which evaluates policies attached to the calling principal. Consequently, a rule that detects whether an IAM policy permits an unintended action under specific conditions cannot be implemented by any mechanism that does not have access to policy data and does not execute the IAM policy evaluation logic.

4-3-9 Evaluation of AWS Native IAM Security Services

AWS services operating at the same control-plane layer, including AWS IAM Identity Center, AWS Config, AWS Security Hub, Amazon GuardDuty, AWS Organizations Service Control Policies, and IAM Permission Boundaries, provide partial overlap with the checks implemented in Phase One, but none offers complete functional equivalence to the Phase One rule catalogue.

AWS IAM Identity Center reduces the need for some Phase One rules by replacing direct authentication with federated access, but it does not address root-account, policy-structure, Attribute-Based Access Control, or credential lifecycle risks.

AWS Config covers only a limited subset of rules and does not produce structured outputs suitable for downstream analysis, while AWS Security Hub aggregates findings without extending detection coverage.

Amazon GuardDuty provides behavioral detection after events occur, whereas Phase One performs proactive analysis of credential posture; the two approaches are therefore complementary.

Service Control Policies and Permission Boundaries enforce constraints and limit the impact of missconfigurations but do not detect them; instead, they are modeled as constraints in Phase Two.

The framework therefore retains all forty-five rules. The Role-Based Access Control rules are included as a grounded and auditable baseline that produces structured findings for downstream analysis. The Attribute-Based Access Control rules capture patterns not collectively addressed by existing open-source IAM analysis tools. No peer-reviewed open-source IAM analyzer, including PMapper, Cloudsplaining, Pacu, TAC [19], and SkyEye [28], detects this combination of patterns. The integration of Phase One findings into Phase Two and Phase Three constitutes a system-level capability not provided by existing AWS-native services.

4-4 Phase Two — Privilege-Graph Implementation

4-4-1 Graph Construction

The graph builder iterates over the unified inventory and emits one node per principal, including IAM users, IAM roles, and IAM groups. Groups are modeled as aggregation nodes rather than independent actors, as AWS does not authorize requests directly at the group level.

Resource-policy modeling for Amazon Simple Storage Service buckets, AWS Key Management Service keys, and AWS Lambda functions is reserved for future work. In the current implementation, edges that would otherwise originate from these resources are inferred from the trust policies of the receiving roles.

Edges are constructed in two passes. The first pass enumerates all privilege-granting actions present in identity-based policies within the inventory. Each candidate edge is classified using an EdgeType enumeration, which includes actions such as ASSUME_ROLE, PASS_ROLE, ATTACH_USER_POLICY, ATTACH_ROLE_POLICY, PUT_USER_POLICY, PUT_ROLE_POLICY, CREATE_ACCESS_KEY, CREATE_POLICY_VERSION, and UPDATE_ASSUME_ROLE_POLICY, as well as the tag-mutation action family.

The second pass evaluates each candidate edge using a policy-evaluation core that reproduces the AWS IAM deny-overrides semantics. For each candidate, the system checks whether the source principal is explicitly denied the corresponding action by examining all applicable policies, including attached, inline, and group-inherited policies. If a matching explicit deny is found, the edge is pruned. Permission boundaries are applied as an additional constraint by intersecting the allowed permissions of each principal, further filtering the resulting set of edges.

4-4-2 IAM Condition Evaluator

The Condition evaluator reads each statement's Condition block and iterates over every operator key (StringEquals, StringNotEquals, StringEqualsIfExists, StringLike, Bool, ArnLike, ArnEquals, Null, ForAllValues:StringEquals, ForAnyValue:StringEquals) for which an explicit handler is defined. The dispatch mechanism supports both the literal operator name and its case-insensitive normalized form. Operator names carrying a ForAllValues: or ForAnyValue: quantifier prefix is first stripped of the prefix, after which the remaining operator is dispatched to the corresponding single-key handler with the appropriate quantifier semantics.

The evaluator returns one of three verdicts per Condition: true, false, or unknown. The unknown verdict is produced whenever an operator without a defined handler is encountered or when the request-context value of a referenced condition key cannot be statically determined from the inventory. In such cases, the candidate edge is marked as conditional rather than admitted unconditionally, preserving soundness in the presence of unsupported operators or incomplete context.

The IfExists family of operators, together with the Null operator, encode the fail-open and fail-closed semantics defined by AWS. These semantics are preserved by explicitly checking for paired Null conditions on the same key whenever an IfExists operator is evaluated.

4-4-3 Tag-Mutation Edges and Joint-State Reachability

Tag-mutation actions are modeled as a distinct edge class. When the graph engine encounters a principal that has any of the iam:TagUser, iam:UntagUser, iam:TagRole, iam:UntagRole, iam:TagPolicy, ec2:CreateTags, ec2:DeleteTags, s3:PutBucketTagging, or s3:DeleteBucketTagging permissions on a target whose tag attributes appear in a downstream *Condition*, the engine emits a STATE_TRANSITION edge that mutates the principal's or the target's tag-state hash.

The reachability search uses a dedicated joint-state traversal strategy. Nodes are visited under a (node identity, tag-state hash) key rather than under node identity alone, allowing the same node to be visited multiple times under different attribute configurations.

The joint-state search is bounded by two parameters: the overall graph hop limit max_hops, with a default value of five, which bounds the total number of edges traversed from any source, and the tag-mutation budget k, with a default value of three, which bounds the number of STATE_TRANSITION edges that the search may take within those five hops. The default value of three for the tag-mutation budget is pre-registered in the project's pre-registration document and reflects the empirical observation that every published path in the IAM Vulnerable benchmark and the Pathfinding.cloud catalog is reachable within three tag-mutation steps.

Figure 9 illustrates the four edges that arise in a small example constructed from these primitives. The principal alice can self-mutate her own tags through iam:TagUser; can pass dev-role to a downstream service through iam:PassRole; can then assume the privileged admin-role through sts:AssumeRole, gated by a Condition requiring the principal to carry the tag Team = admin; and the assumed admin-role can read the prod-data S3 bucket through s3:GetObject.

Without the tag-mutation edge, the AssumeRole condition fails and the path is closed. The joint-state breadth-first search visits alice once with the original tag set and a second time after the self-loop is traversed, thereby exposing the escalation that a permission-only graph would miss.

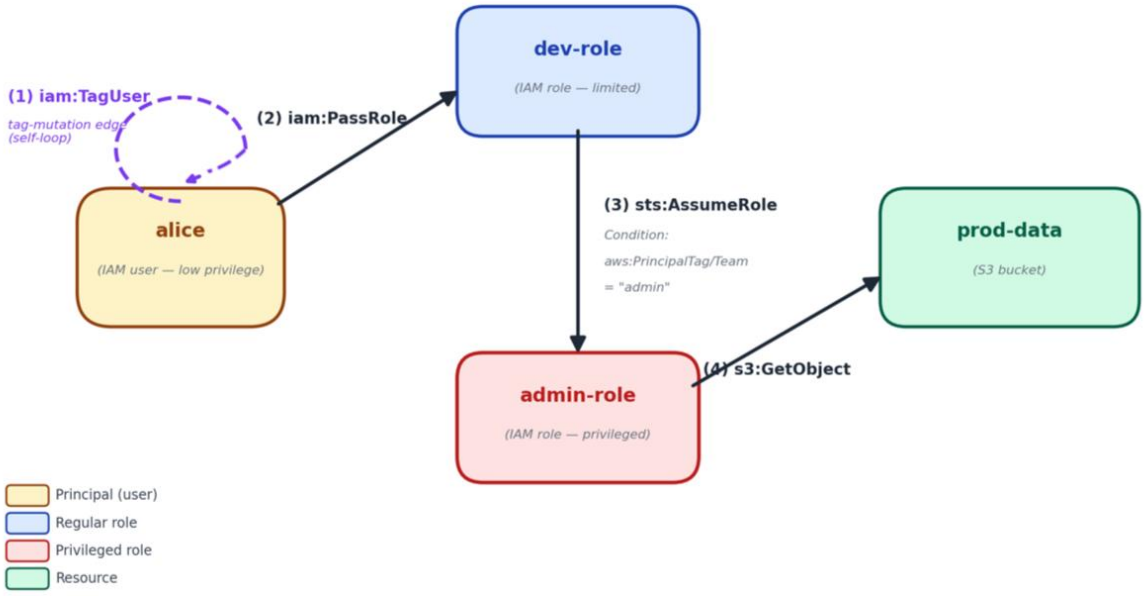

Figure 9: Illustrative Phase Two Privilege-Escalation Graph — A Tag-Mutation Path from a Low-Privilege User to a Privileged Resource

4-4-4 Bounded Breadth-First Search

The path engine exposes two breadth-first traversal functions. The first performs conventional permission-only privilege-graph traversal and is used to discover paths that do not require attribute mutation. The second performs the joint-state traversal described in the previous subsection and is used for tag-mutation paths. Both traversals are bounded by max_hops, with a default value of five. The joint-state variant is additionally bounded by the tag-mutation budget k, with a default value of three.

Each traversal terminates a path when the breadth-first search reaches a node that has been flagged as is_admin by the graph builder. The admin-flagging routine uses two complementary detection strategies: an admin-by-document check, which scans all effective policy documents, including attached, inline, and group-inherited policies, for an Allow effect on the wildcard action over the wildcard resource; and an admin-by-policy-name shortcut, which recognizes the AWS-managed AdministratorAccess policy by its Amazon Resource Name, even when its document is not fetched into the inventory.

Each discovered attack path is annotated with the source principal, the sequence of traversed edges, the EdgeType of each edge, the encountered Condition verdicts, and any tag-state mutations performed along the path.

4-4-5 Service Control Policy and Permission Boundary Filtering

The Service Control Policy (SCP) filter applies organization-level guardrails to every candidate path after the breadth-first search produces it. The filter traverses the path's edge sequence and, for each edge action, evaluates the SCP hierarchy from the organization root down to the calling account using the same deny-overrides semantics as the IAM evaluator. A path is annotated as scp_blocked when any SCP in the hierarchy denies the action, and as scp_evaluated_clean when every SCP either explicitly allows or does not restrict the action.

The Permission Boundary filter operates as a per-principal intersection mask during graph construction rather than as a post-filter on complete paths. When a principal has an iam:PermissionsBoundary attached, the boundary policy's permitted action set is intersected with the principal's effective identity-policy permissions. Any candidate edge that depends on an action outside this intersection is pruned at construction time.

This separation between post-processing SCP evaluation and construction-time Permission Boundary enforcement reflects the difference in scope between the two mechanisms: SCPs apply at the account level, whereas Permission Boundaries apply at the individual principal level.

4-5 Phase Three — Correlation and Grounded Reasoning Implementation

4-5-1 CloudTrail Event Ingestion and Normalization

The CloudTrail ingestion module, implemented in cloudtrail_normalizer.py, reads the events emitted by the data-collection layer and produces a list of NormalizedEvent objects through the CloudTrailNormalizer.normalize_file or normalize_list methods. Each raw event's userIdentity field is parsed by the helper canonical_principal_type, which collapses the AWS-documented userIdentity types (Root, IAMUser, AssumedRole, AWSService, FederatedUser, AWSAccount, Anonymous, Unknown) into a small PrincipalType enum that the downstream correlator uses for matching. The normaliser additionally extracts a per-event target identifier (the role Amazon Resource Name, user Amazon Resource Name, or resource Amazon Resource Name that the event acts on) from the requestParameters, and a per-event tag key and value when the event represents a tag-mutation action. Events whose caller Amazon Resource Name belongs to the running operator (obtained through the AWS Security Token Service GetCallerIdentity call at framework startup) are dropped by the helper _is_operator_event, which prevents the framework's own read-only inventory traffic from contaminating the analysis.

4-5-2 Deterministic MITRE ATT&CK Mapping

The MITRE ATT&CK mapper, implemented in mitre_mapper.py, applies a deterministic-first lookup whose primary table maps each CloudTrail eventName to a tuple of (tactic, tactic identifier, technique, technique identifier, sub-technique, sub-technique identifier) sourced from the public ATT&CK for Cloud (IaaS) matrix. The mapping table is curated rather than learned: each entry is populated by hand from the ATT&CK technique pages, and any event whose name has no entry returns an UNMAPPED record rather than a guessed assignment. A secondary mapping basis exists for events that can be refined through Phase Two context — for example, an event whose event-name lookup is ambiguous but whose Phase Two correlation places it on a PASS_ROLE edge can be refined to the corresponding sub-technique — and the resulting mapping carries a mapping_basis field whose value is HIGH for direct event-name lookup, MEDIUM for edge-type contextual mapping, or NONE for UNMAPPED. The deterministic property of the mapping is what allows every alert to carry a verifiable technique identifier independent of the Large Language Model, and the explicit UNMAPPED fallback is what allows the framework to refuse to fabricate a label when no grounded mapping exists.

4-5-3 Correlation with Phase One and Phase Two Outputs

The correlation engine, implemented in correlator.py, ingests the Phase One findings through phase1_loader.py and the Phase Two attack paths through phase2_loader.py, and matches each NormalizedEvent against both inputs. Matching is principal-keyed: the event's caller Amazon Resource Name is compared against the principal Amazon Resource Names recorded on every Phase One finding and on every Phase Two path, and a match is reported when the two coincide. Each correlated event is then assigned to one of four correlation tiers. STRONGLY_CORRELATED is assigned when both a Phase One finding at high or critical severity and a Phase Two path are present for the same principal. PARTIALLY_CORRELATED is assigned when only one of the two is present. DYNAMIC_STANDALONE is assigned when no prior-phase match is found but the event's mapped MITRE technique is in a curated MITRE-grounded sensitive-action catalog. STATIC_STANDALONE is assigned when a Phase One finding exists for the principal but no CloudTrail event corroborates it. The standalone catalog is itself sourced from ATT&CK procedure-linked actions rather than from a practitioner heuristic, and each entry carries an event_sensitivity field of HIGH or MEDIUM that is derived from the mapped tactic family but is explicitly distinguished from the overall alert severity. Overall severity is reserved for correlated findings where threat context is established jointly from Phase One and Phase Two.

4-5-4 Knowledge Base and Vector Index

The knowledge base is a curated collection of authoritative passages drawn from the MITRE ATT&CK technique pages, the AWS service-authorization references, the AWS IAM Action Reference, the CIS AWS Foundations Benchmark control descriptions, and the AWS official remediation playbooks. The knowledge-base build utility chunks each source document, computes an embedding through a locally-served sentence-embedding model, and writes the resulting embedding matrix to phase3/knowledge/kb_index/kb.faiss alongside a parallel kb_metadata.json file that preserves each chunk's source attribution, category (authoritative or supplemental), related action list, and an authoritative Boolean flag. The retriever in retriever.py loads the FAISS index lazily, computes the query embedding, and returns the top-k chunks ranked by cosine similarity

over the normalised embedding vectors. Source category is preserved through retrieval but does not influence ranking, so the retriever returns only what is most semantically similar to the query rather than biasing toward authoritative-flagged chunks. When the FAISS index is not present on disk the retriever degrades gracefully by returning an empty result list, which the Large Language Model adapter then handles by passing the model only the deterministic correlation evidence.

4-5-5 Large Language Model Adapter and Two Backends

The Large Language Model adapter, implemented in llm_adapter.py, exposes a single LLMAdapter.generate interface that supports two backends in the present configuration. The first backend is a deterministic null backend identified by the string 'none' that performs no generation and serves as the no-LLM experimental control: under this backend, every Large-Language-Model field on the resulting alert is left empty, so the alert contains only the deterministic correlation tier, the MITRE ATT&CK technique mapping, the evidence identifiers, and the retrieved knowledge passages, and serves as the lower bound against which any LLM-augmented configuration is measured. The second backend is an OpenAI-compatible client identified by the string 'openai_compatible' that talks to any inference endpoint conforming to the OpenAI Application Programming Interface; the default runtime path uses this backend with the API base set to the loopback Ollama daemon and the model identifier set to llama-3-1-8b-instruct, which serves the general-purpose Llama-3.1-8B-Instruct model in the Q4_K_M quantisation. The temperature is pinned to zero in every evaluation run, and every alert records the backend identifier and the served model name through a dedicated metadata field, which permits direct experimental comparison of the no-LLM baseline against the general-purpose model under identical evidence and prompt conditions. The substitution of a cybersecurity-specialised Large Language Model in place of the general-purpose backend is reserved for future work.

4-5-6 Hallucination Guard

The hallucination guard, implemented in hallucination_guard.py, validates every Large Language Model output against the EvidenceBundle assembled by the correlator. The guard's HallucinationGuard.validate method begins by short-circuiting when the backend identifier is 'none': for the deterministic null backend no LLM claims exist to validate, so the guard returns a NOT_APPLICABLE GroundingReport rather than treating empty output as ungrounded — without this distinction, the no-LLM baseline would be reported as ungrounded alongside actual hallucinations, which is the opposite of what the metric should measure. For the OpenAI-compatible backend, the guard validates four classes of factual claim against the evidence bundle. First, every evidence_ids_referenced entry must appear in the bundle's known evidence-identifier set. Second, every MITRE tactic in the LLM output must appear in the bundle's known-tactic list. Third, every MITRE technique identifier must match the curated regular expression for valid ATT&CK identifiers and must appear in the bundle's known-technique list. Fourth, every Amazon Resource Name in the LLM's free-form text is scanned for phantom Amazon Resource Names that are not present anywhere in the input evidence. Outputs that fail any of these four checks are discarded and replaced with a deterministic alert that contains only the evidence and the deterministic MITRE mapping, which guarantees that no alert delivered to the responder contains unsupported claims.

4-6 Evaluation Datasets and Ground-Truth Catalogs

4-6-1 IAM Vulnerable Benchmark

IAM Vulnerable, published by BishopFox [6], deploys a set of intentionally vulnerable IAM configurations into a real AWS account through Terraform. The deployment creates twenty-one in-scope privilege-escalation path classes that Cloud IAM Sentinel must detect to be considered effective. The benchmark is the primary detection-rate target of the framework's Phase Two evaluation.

4-6-2 Pathfinding.cloud Catalog

The Pathfinding.cloud catalog, published by Datadog Security Labs [7], enumerates sixty-six privilege-escalation paths across AWS, Azure, and GCP, each described by a structured schema that lists the prerequisite permissions and the expected edge sequence. The catalog is used to seed the privilege-granting edge types modeled by the framework's Phase Two and to verify that every published path is representable in the framework's privilege graph.

4-6-3 CloudGoat Scenarios

CloudGoat, published by Rhino Security Labs, is an open-source vulnerable AWS deployment that includes a small set of IAM-only privilege-escalation scenarios. The framework's evaluation harness includes a CloudGoat ground-truth file at eval/cloudgoat_ground_truth.yaml and a scoring script at eval/scripts/score_phase2_against_cloudgoat.py; the actual exercise of the framework against the deployed CloudGoat scenarios is staged for the Chapter Five evaluation campaign and has not been performed in the present report.

4-6-4 Stratus Red Team Attack Catalog

The Stratus Red Team adversary-emulation framework, published by Datadog Security Labs, deterministically reproduces a catalog of cloud attacks against a controlled tenant. The framework's evaluation harness includes a Stratus ground-truth file at eval/stratus_ground_truth.yaml and a scoring script at eval/scripts/score_phase3_against_stratus.py; the actual exercise of the framework against Stratus-emitted CloudTrail events is staged for the Chapter Five evaluation campaign and has not been performed in the present report.

4-6-5 flaws.cloud CloudTrail Dataset

The flaws.cloud CloudTrail dataset is a public release of approximately 1.9 million CloudTrail events captured from a benign AWS account over a multi-year period. The dataset is intended to serve as the framework's false-positive baseline: any alert emitted against it is, by construction, a false positive that the hallucination guard or the correlation tier failed to suppress. The framework's evaluation harness includes a fetch script at eval/scripts/fetch_flaws_cloud_sample.py and a scoring script at eval/scripts/score_phase3_fpr_against_flaws_cloud.py; the actual exercise of the framework against the flaws.cloud dataset is staged for the Chapter Five evaluation campaign and has not been performed in the present report.

4-7 Comparison Tools

The framework is evaluated head-to-head against the open-source IAM analysis tools introduced in Chapter Two: Principal Mapper, Cloudsplaining, ScoutSuite, Prowler, and SkyEye [28]. Each tool is installed on the same evaluation host as Cloud IAM Sentinel, run against the same target AWS account using the same read-only permission policy, and its output is normalized into a common Comma-Separated Value schema that records the principal, the target privilege, the path or finding identifier, the matched MITRE ATT&CK technique if any, and the tool's confidence annotation if any. The normalized outputs are then compared against the ground-truth annotations of the datasets in Section 4-6, which is the methodology used by the experiments reported in Chapter Five.

4-8 Conclusion

This chapter described the practical implementation of Cloud IAM Sentinel: the programming environment and the read-only AWS permission policy that scope every interaction with the cloud account; the forty-five rules of Phase One organized into six detection families, with an explicit description of the iteration scaffolding, the shared pattern-detection helpers, and the violation-record schema that each family follows in code; the privilege-graph implementation of Phase Two, including the EdgeType enumeration, the deny-overrides candidate-edge admission loop, the ten-operator Condition evaluator, the STATE_TRANSITION tag-mutation edge class, the joint-state breadth-first search bounded by a tag-mutation budget of three within an overall hop limit of five, and the split between post-filter Service Control Policy handling and at-construction-time Permission Boundary handling; the correlation and grounded-reasoning implementation of Phase Three, including the userIdentity normalizer, the deterministic-first MITRE ATT&CK mapper with its three mapping bases, the four correlation tiers (strongly correlated, partially correlated, dynamic standalone, and static standalone), the FAISS-backed knowledge retrieval with graceful fallback, the two-backend Large Language Model adapter, and the four-class hallucination guard; the open ground-truth datasets used by the evaluation harness, with an honest distinction between datasets that have been exercised in the present report and datasets that are staged for the Chapter Five evaluation campaign; and the open-source comparison tool installed alongside the framework. The next chapter reports the results of the evaluation campaign that exercises this implementation against the IAM Vulnerable benchmark, the Pathfinding.cloud catalogue, the staged datasets, and the head-to-head comparison tools.

Chapter Five

Evaluation

5-1 Introduction

This chapter reports the evaluation of Cloud IAM Sentinel against externally curated benchmarks and end-to-end attack scenarios. The evaluation is structured around two complementary axes: a per-phase axis that measures each phase against an external comparator and ground-truth corpus, and an integrated axis that exercises the entire pipeline on staged attack scenarios deployed in a dedicated Amazon Web Services account. The chapter opens with a statement of the evaluation methodology that governs the entire chapter, including the principle of comparator-based decision rules, the use of confidence intervals on every reported proportion, and the rejection of evaluator-authored numeric thresholds. The chapter then presents the Phase One results against the BishopFox IAM Vulnerable adversarial principal set and against the Cloudsplaining policy classifier on the same deployed account, the Phase Two results against the Datadog Pathfinding.cloud sixty-six-path catalog and against the Bishop Fox two-thousand-and-twenty-one thirty-one-path benchmark, and finally the per-scenario walkthroughs of two CloudGoat scenarios that exercise the entire pipeline end-to-end.

5-2 Evaluation Methodology

The evaluation rests on three principles. The first principle is two layers of evidence per phase. Each phase is evaluated both in isolation against an external ground-truth corpus and as a component of the integrated pipeline on staged attack scenarios. The phase-alone evaluation establishes that each component performs against a comparator that is independent of the framework; the integrated evaluation establishes that the complete pipeline performs end-to-end on attack traffic that the framework did not see during development.

The second principle is descriptive metrics with confidence intervals and paired statistical comparisons against external comparators, in place of evaluator-authored numeric thresholds. Every numeric pass-fail criterion in the pre-registration is stated as a relational claim, such as detecting strictly more catalog paths than a named comparator or producing a paired-significantly lower false-positive rate than the framework's own deterministic-only mode. No fixed numeric threshold of the form greater than or equal to a specified percent recall, or less than or equal to a specified percent false-positive rate, is authored by this work; comparators are externally published baselines or the framework's own non-augmented mode.

The third principle is externally authored ground truth, with reproducible verification. Every ground-truth corpus is a derivative of an open-source, version-controlled, citable artifact published independently of this work. Each ground-truth file carries a source attribution and a date verified against the upstream release; for two of the four corpora used, an automated drift verifier exits non-zero when the upstream artifact changes. Statistical tests follow the established conventions: confidence intervals on proportions are the Wilson score interval [12]; paired binary outcomes are tested with McNemar's exact test [12]; multiple comparisons are corrected with the Bonferroni adjustment at family-wise alpha equal to zero point zero five [12]; non-parametric confidence intervals on rate differences are computed by percentile bootstrap with one thousand resamples and a fixed seed [12].

5-3 Phase One - Static IAM Posture Evaluation

Phase One implements forty-one active rules derived from the Center for Internet Security Amazon Web Services Foundations Benchmark version three point zero point zero, the Amazon Web Services Foundational Security Best Practices catalog, the National Institute of Standards and Technology Special Publication eight hundred dash fifty-three Revision Five control baseline, and the National Institute of Standards and Technology Special Publication eight hundred dash one hundred sixty-two attribute-based access control guidance. Phase One is evaluated under two complementary metrics. The first metric measures adversarial accuracy on the nine adversarial principals deployed by the BishopFox IAM Vulnerable free-resources Terraform module. The second metric measures coverage of the per-principal verdict vector against the Cloudsplaining policy classifier, run on the same deployed account.

5-3-1 Adversarial Accuracy on Nine Adversarial Principals

BishopFox IAM Vulnerable [6] deploys nine principals authored specifically to exercise the correctness of static IAM analyzers. Five of these principals are false-positive cases whose effective permissions are scoped away by Deny statements, never-matching Resource constraints, or Conditions that never evaluate true; a correctly implemented analyzer must not classify them as exploitable. Four of these principals are false-negative cases whose effective permissions are reachable despite surface-level filtering by NotAction, exploitable Conditions, or partial-permission cross-principal collaboration; a correctly implemented analyzer must classify them as exploitable. The fp/fn correctness matrix carries fourteen pre-committed (principal, rule) verdict pairs.

Phase One was run against the deployed IAM Vulnerable account and the verdicts were tabulated. Eleven of the fourteen pairs received a strictly correct verdict; three received an acceptable over-approximation, which corresponds to documented static-analysis limitations under undecidable Conditions and Resource constraints; no pair received an incorrect verdict. The Wilson ninety-five percent confidence interval on the strictly correct count is fifty-two point four percent through ninety-two point four percent. The Wilson ninety-five percent confidence interval on the incorrect count is zero percent through twenty-one point five percent.

Verdict class	Count	Proportion	Wilson 95 % CI
Strictly correct	11 / 14	78.6 %	[52.4 %, 92.4 %]
Acceptable over-approximation	3 / 14	21.4 %	[7.6 %, 47.6 %]
Incorrect	0 / 14	0.0 %	[0.0 %, 21.5 %]

Table 18: Phase One adversarial-accuracy verdict counts on the nine BishopFox IAM Vulnerable principals (n = 14 pairs).

The result satisfies the pre-registered Win-condition clause on the adversarial-accuracy axis: zero incorrect verdicts on the fourteen pairs the ground truth marks as in scope, and three acceptable over-approximations explicitly recognized as documented static-analysis limits. Phase One correctly applies the Deny-overrides-Allow rule on the fp1, fp2, and fp3 principals, correctly classifies the fn4 NotAction broadening case as a violation, and conservatively over-approximates on the two undecidable cases that the static analyzer cannot soundly resolve without runtime context.

5-3-2 Coverage Versus Cloudsplaining on the Per-(Check, Principal) Detection Vector

Phase One's per-(check, principal) verdict vector was compared against Cloudsplaining's per-policy risk classifications, mapped to the Phase One check identifiers via a conservative authority-grounded mapping table. The mapping table includes thirty Phase One checks. Twenty have a peer in the Prowler IAM check family and nine have a peer in one of Cloudsplaining's six risk categories; ten checks have no peer in either tool and are reported as out-of-comparison. Cloudsplaining produces five hundred and ten fired (check, principal) pairs across the six risk categories on the deployed account; Phase One produces fifty-eight fired pairs across its forty-one checks.

Restricted to the eight hundred and ten (check, principal) pairs whose Phase One check has a Cloudsplaining peer in the mapping table, the contingency table records ten pairs in which Phase One fires and Cloudsplaining does not, seven hundred and sixty pairs in which Cloudsplaining fires and Phase One does not, five pairs in which both tools fire, and thirty-five pairs in which neither tool fires. McNemar's exact two-sided p-value is below zero point zero zero zero one. The result rejects the hypothesis that the two tools produce identical detection vectors on the (check, principal) pairs they share.

Cell	Count
Only Phase One fires	10
Only Cloudsplaining fires	760
Both fire	5
Neither fires	35
Total (check, principal) pairs	810

Table 19: Phase One versus Cloudsplaining 2x2 contingency table on the per-(check, principal) detection vector.

The directional asymmetry observed in the contingency table reflects the two tools' different design objectives. Cloudsplaining classifies every customer-managed and inline policy under any combination of six broad risk categories whenever the policy text matches the corresponding risk pattern. A single policy that grants any IAM permission may therefore be flagged in multiple risk categories simultaneously, and the categories propagate to every principal the policy is attached to. Phase One's per-check rules, by contrast, are bound to specific authoritative controls - root-account hygiene, multi-factor authentication on console access, password policy, and the privilege-escalation primitives identified in the Rhino Security Labs taxonomy - and emit a single FAIL per (check, violator). The two designs measure overlapping but non-equivalent properties of the same account, and the paired test reflects that overlap-without-equivalence honestly.

The pre-registration's Win-condition clause on the coverage-delta axis is met. Combining the two metrics, Phase One reaches the Win outcome on this evaluation: zero incorrect verdicts on the adversarial principal set, and a paired-significant detection-vector difference against the available comparator.

5-4 Phase Two - Privilege-Escalation Graph Evaluation

Phase Two is evaluated on three independent ground-truth corpora to mitigate single-corpus bias. The primary corpus is the Datadog Pathfinding.cloud catalog [7], a sixty-six-path open enumeration of Amazon Web Services privilege-escalation paths fetched at upstream commit f7170aff. The secondary corpus is the BishopFox IAM Vulnerable benchmark [6], with twenty-one in-scope path classes and a thirty-one-path full denominator that aligns with the published baseline numbers reported by Bishop Fox in two thousand and twenty-one. The tertiary corpus is the CloudGoat scenario library, covered in Section 5-5 of this chapter.

5-4-1 Pathfinding.cloud Catalog as Primary Corpus

Two detection criteria are reported. The loose criterion counts a catalog path as detected if Phase Two produces an attack path from the catalog's source principal that reaches admin. The strict criterion additionally requires the documented action sequence to appear as an ordered subsequence of the framework's edge-type list. Comparator detection vectors for PMapper, Cloudsplaining, and Pacu are taken from the catalog's published detection-tools attribution field; the framework is therefore compared against published attribution rather than against run-time output of the comparator tools.

Phase Two detected forty-eight of sixty-six paths under the loose criterion and forty-five of sixty-six paths under the strict criterion. The catalog-attribution counts for the comparator tools are PMapper twenty paths, Cloudsplaining nineteen paths, and Pacu nineteen paths. Wilson ninety-five percent confidence intervals are reported on every Phase Two proportion. Six paired McNemar's exact tests against the three IAM-focused comparators are computed at family-wise alpha equal to zero point zero five with Bonferroni correction (k equal to three; alpha-prime equal to zero point zero one six seven). Every paired test reaches Bonferroni-corrected significance on both criteria.

Detector	Loose detected	Strict detected	Loose Wilson 95 % CI
Cloud IAM Sentinel (this work)	48 / 66 (72.7 %)	45 / 66 (68.2 %)	[61.0 %, 82.0 %]
PMapper (catalog attribution)	20 / 66 (30.3 %)	20 / 66 (30.3 %)	-
Cloudsplaining (catalog attribution)	19 / 66 (28.8 %)	19 / 66 (28.8 %)	-
Pacu (catalog attribution)	19 / 66 (28.8 %)	19 / 66 (28.8 %)	-

Table 20: Phase Two detection on the sixty-six-path Pathfinding.cloud catalog at upstream commit f7170aff.

Comparator	Criterion	p-value	Significant after Bonferroni
PMapper	loose	< 0.0001	yes
PMapper	strict	0.0002	yes
Cloudsplaining	loose	< 0.0001	yes
Cloudsplaining	strict	< 0.0001	yes
Pacu	loose	< 0.0001	yes
Pacu	strict	< 0.0001	yes

Table 21: Paired McNemar's exact tests against PMapper, Cloudsplaining, and Pacu on the Pathfinding.cloud detection vector.

The Phase Two pre-registration Win condition on the primary corpus is met. Phase Two detects strictly more catalog paths than each of the three IAM-focused comparators on the catalog's own attribution vector, with paired-significance after Bonferroni correction.

5-4-2 IAM Vulnerable Benchmark as Secondary Corpus

On the twenty-one in-scope COVERED path classes documented in the IAM Vulnerable ground-truth file, Phase Two achieves a recall of ninety-five point two percent (twenty of twenty-one), with a Wilson ninety-five percent confidence interval of seventy-seven point three through ninety-nine point two percent. When the recall denominator is restricted to the nineteen COVERED classes whose principal-and-trust-policy topology is actually instantiated by the deployed free-resources Terraform subset, Phase Two achieves a recall of one hundred percent, with a Wilson ninety-five percent confidence interval of eighty-three point two through one hundred percent.

On the thirty-one-path Bishop Fox two-thousand-and-twenty-one full denominator, Phase Two detects twenty paths (sixty-four point five percent), with a Wilson ninety-five percent confidence interval of forty-six point nine through seventy-eight point nine percent. Phase Two ranks third on this corpus, two paths behind PMapper at twenty-two paths and one path behind Pacu at twenty-one paths, and one path ahead of Cloudsplaining at nineteen paths.

Detector	Detected	Proportion	Wilson 95 % CI
PMapper	22 / 31	71.0 %	-
Pacu	21 / 31	67.7 %	-
Cloud IAM Sentinel (this work)	20 / 31	64.5 %	[46.9 %, 78.9 %]
Cloudsplaining	19 / 31	61.3 %	-
AWSPX	14 / 31	45.2 %	-

Table 22: Phase Two detection on the thirty-one-path IAM Vulnerable benchmark (Bishop Fox 2021 denominator).

The two corpora measure different things and produce different rankings. The two-thousand-and-twenty-one Bishop Fox benchmark predates widespread attribute-based access-control adoption and is the dataset PMapper was originally tuned against; its eleven deferred version-one non-goals are paths that no version-one tool was expected to detect by the original benchmark's design. The Pathfinding.cloud catalog is current, includes attribute-based access-control paths, and reports a per-path attribution graph that lets the framework be paired against the comparator tools using the catalog's own ground truth. Cloud IAM Sentinel substantially outperforms each comparator on the more current and broader benchmark; on the legacy benchmark the framework is competitive but ranks third by two paths behind PMapper.

5-5 End-to-End Validation on CloudGoat Scenarios

The CloudGoat scenario library is used to exercise the entire pipeline end-to-end. Each scenario is deployed in isolation in the evaluation account, the framework's three phases are run against the

deployed account, the documented attack chain is executed using the scenario's starting credentials, the resulting CloudTrail events are collected, and Phase Three is run on the captured events under both the deterministic-only baseline and the Large-Language-Model-augmented baseline. The per-scenario outputs are filtered to only the alerts whose principal or affected resource is within the scenario's deployed in-scope identifier set, so the per-scenario results are not contaminated by activity unrelated to the scenario. After each scenario's outputs are saved, the deployed resources are destroyed before the next scenario is deployed, ensuring the scenarios do not interact with one another.

Two scenarios are reported in this section. The first is iam_enum_basics, which serves as the negative control: an attacker with low-privileged credentials performs read-only IAM enumeration and does not exercise any privilege-escalation primitive. The second is iam_privesc_by_key_rotation, an attribute-based access-control privilege-escalation scenario in which a low-privileged user mutates the tag attribute on a higher-privileged user to satisfy a previously unsatisfied tag-conditioned permission, and subsequently rotates the higher-privileged user's access keys. The two further CloudGoat scenarios, the Phase-Three evaluation against the Stratus Red Team catalog, the false-positive-rate evaluation against the flaws.cloud public dataset, and the integrated false-positive-rate comparison are presented in subsequent sections after the corresponding evaluation runs are complete.

5-5-1 Scenario One: iam_enum_basics (Negative Control)

The iam_enum_basics scenario is the negative control. The scenario deploys one IAM user named cg-bob, one IAM group with an unusual hierarchical path, one IAM role assumable by cg-bob, one customer-managed policy, and one inline policy attached to cg-bob. The user cg-bob carries the Amazon Web Services-managed IAMReadOnlyAccess policy and an inline policy that grants ec2:DescribeInstances on all resources. The deployed user has no permission to write any IAM object and no permission to assume any role with elevated privileges; the attack chain consists of five read-only IAM Application Programming Interface calls used to retrieve five flags hidden in the metadata of the deployed resources. The expected framework behavior is therefore the absence of any privilege-escalation evidence and the absence of correlated-tier promotion in Phase Three.

Phase One retains one finding for this scenario after per-scenario filtering: iam_policy_attached_only_to_group_or_roles, with severity MEDIUM, anchored in the Center for Internet Security Foundations Benchmark version three control one point fifteen. The finding identifies cg-bob as a user with a managed policy attached directly to the user account rather than via a group; this is a posture observation about access-control hygiene, not a privilege-escalation claim.

Figure 23 presents the JavaScript Object Notation representation of the Phase One finding emitted on this scenario.

```
{
  "check_id": "iam_policy_attached_only_to_group_or_roles",
  "status": "FAIL",
  "severity": "MEDIUM",
  "severity_basis": "CIS AWS Foundations Benchmark v3.0.0 Control 1.15",
  "status_extended":
      "Users with direct policy attachments: cg-bob-cgidg0dl1n7zeh",
  "evidence": {
    "details": {
      "violations": [
        {
          "resource_type": "AWS::IAM::User",
          "resource_id":
              "arn:aws:iam::344809605381:user/cg-bob-cgidg0dl1n7zeh",
          "name": "cg-bob-cgidg0dl1n7zeh",
          "detail": { "attached_policy_count": 2 }
        }
      ]
    }
  }
}
```


Figure 23: Phase One finding on the iam_enum_basics scenario actor.

Phase Two retains zero attack paths from any of this scenario's principals, which is the expected and correct behavior because cg-bob has read-only IAM permissions only and no privilege-escalation primitive is exercised by the deployed account. CloudTrail capture during attack execution recorded one thousand and forty-six Identity-and-Access-Management-relevant events across the lookback window; events generated by the framework's own scanning identity were excluded from the Phase Three input.

Phase Three retains nine alerts for this scenario after per-scenario filtering. Zero alerts reach the STRONGLY_CORRELATED tier, zero alerts are typed as ACTIVE_EXPLOITATION, and nine alerts are typed as LATENT_RISK at the PARTIALLY_CORRELATED tier. The framework correctly does not promote any alert to the strict-correlation tier, because no Phase Two attack path exists from the actor. The PARTIALLY_CORRELATED tier reflects the framework's correlation of cg-bob's read-only Application Programming Interface calls with the Phase One CIS one point fifteen finding; this is a posture-warning category, not a privilege-escalation claim.

Figure 24 presents the JavaScript Object Notation representation of a Phase Three alert produced under the Large-Language-Model-augmented baseline for the GetRole event by cg-bob, in which the Large Language Model has produced grounded explanation, priority justification, false-positive assessment, and remediation fields.

```
{
  "alert_id": "f8d2d35838e6",
  "alert_type": "LATENT_RISK",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "source_cloudtrail_event_names": ["GetRole"],
  "principal_arns":
      ["arn:aws:iam::344809605381:user/cg-bob-cgidg0dl1n7zeh"],
  "mitre": {
    "tactic": "Discovery",
    "technique": "Cloud Account Discovery",
    "technique_id": "T1087.004"
  },
  "llm_explanation":
      "The principal cg-bob-cgidg0dl1n7zeh has direct policy attachments,
       which may indicate a security risk
       (check_id=iam_policy_attached_only_to_group_or_roles).",
  "llm_priority_justification":
      "Medium priority due to the presence of direct policy attachments
       and potential privilege escalation risks.",
  "llm_false_positive_assessment":
      "Unlikely, as direct policy attachments are a known security risk
       and may indicate malicious activity.",
  "llm_remediation_steps":
      "Review IAM policies to ensure they are attached correctly,
       without direct policy attachments.",
  "llm_backend_used": "llama-3-1-8b-instruct",
  "grounding_valid": true
}
```


Figure 24: Phase Three alert on the iam_enum_basics scenario, Large-Language-Model-augmented baseline.

The negative-control verdict is therefore a Pass on the strict-correlation criterion. Zero alerts reach the STRONGLY_CORRELATED tier; zero alerts are typed as ACTIVE_EXPLOITATION. The nine PARTIALLY_CORRELATED alerts are the framework's expected behavior when an actor with a real Phase One posture finding generates Identity-and-Access-Management CloudTrail activity, and they are tagged as LATENT_RISK rather than ACTIVE_EXPLOITATION.

5-5-2 Scenario Two: iam_privesc_by_key_rotation (V3 ABAC Re-Tagging)

The iam_privesc_by_key_rotation scenario exercises the attribute-based access-control re-tagging vulnerability class. The scenario deploys a low-privileged user named manager whose policies grant the iam:CreateAccessKey and iam:DeleteAccessKey actions only on users tagged developer equal true. The same user has the iam:TagUser action with no condition. The privileged target is an admin user that can assume a role granting access to the scenario's secret. The attack chain consists of mutating the tag attribute on the admin user, deleting an inactive access key on the admin user to free a slot, and creating a new access key on the admin user, which the attacker then possesses.

Phase One retains two findings for this scenario after per-scenario filtering. The first finding is the same posture observation as in scenario one (CIS one point fifteen on the manager user). The second finding, iam_tag_write_without_tagkeys_restriction with severity HIGH, is the framework's vulnerability-class detector for ResourceTag re-tagging and is the privilege-

escalation primitive detector for this scenario. The detector inspects the manager user's TagResources inline policy, finds a statement that grants the iam:TagUser action on Resource colon asterisk without an aws:TagKeys condition, and flags this as a documented attack primitive [4]. The detector identifies exactly which principal, which policy, and which statement triggered the rule, providing forensic-grade attribution.

Figure 25 presents the JavaScript Object Notation representation of the V3 attribute-based access-control detector firing on the scenario two actor.


```
{
  "check_id": "iam_tag_write_without_tagkeys_restriction",
  "status": "FAIL",
  "severity": "HIGH",
  "severity_basis":
      "AWS IAM documentation - Controlling access using tag keys
       (aws:TagKeys); NIST SP 800-162 §2.4.2; NIST SP 800-53 Rev. 5
       AC-3. Tag manipulation is a documented ABAC privilege-
       escalation vector.",
  "status_extended":
      "1 policy statement(s) grant IAM tag write actions on Resource:*
       without aws:TagKeys conditions, enabling ABAC privilege
       escalation via tag manipulation.",
  "evidence": {
    "details": {
      "violations": [
        {
          "resource_type": "AWS::IAM::User",
          "resource_id":
              "user/manager_cgid2ztrtw8gc5/inline/TagResources",
          "name": "manager_cgid2ztrtw8gc5",
          "detail": {
            "principal_name": "manager_cgid2ztrtw8gc5",
            "policy_type": "inline",
            "policy_name": "TagResources",
            "statement_index": 0,
            "reason":
                "IAM tag write actions on Resource:* without
                 aws:TagKeys condition - permits arbitrary tag
                 manipulation"
          }
        }
      ]
    }
  }
}
```


Figure 25: Phase One V3 attribute-based access-control detector firing on the iam_privesc_by_key_rotation actor.

Phase Two retains zero attack paths from any of this scenario's principals after per-scenario filtering. The scenario's privileged target is the cg_secretsmanager role, which grants secretsmanager:GetSecretValue on a single secret rather than admin-equivalent access. Phase Two's breadth-first reachability search enumerates paths to admin-equivalent destinations only, by

design; lateral access to a single secret is outside the scope of the breadth-first solver. The attack primitive is detected by the Phase One V3 detector instead, which is the framework's complementary static detector for attribute-based access-control patterns that the breadth-first search does not target. The Phase Two silence on this scenario is a documented design choice, not a missed detection.

Phase Three retains eight alerts for this scenario after per-scenario filtering. Zero alerts reach the STRONGLY_CORRELATED tier, because Phase Two produced no attack path on this scenario. Eight alerts are typed as ACTIVE_EXPLOITATION at the PARTIALLY_CORRELATED tier. The framework correlates the manager user's runtime CloudTrail events - TagUser, DeleteAccessKey, and CreateAccessKey - with the V3 attribute-based access-control finding from Phase One and tier-promotes the alert. The deterministic MITRE ATT&CK mapping for the headline CreateAccessKey event is T1098.001 Additional Cloud Credentials, the exact ATT&CK sub-technique for the rotated-credential attack pattern [11].

Figure 26 presents the JavaScript Object Notation representation of the Phase Three Large-Language-Model-augmented alert on the headline CreateAccessKey event by manager.


```
{
  "alert_id": "a8bea45b6f92",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "source_cloudtrail_event_names": ["CreateAccessKey"],
  "principal_arns":
      ["arn:aws:iam::344809605381:user/manager_cgid2ztrtw8gc5"],
  "source_phase1_check_ids": [
    "iam_policy_attached_only_to_group_or_roles",
    "iam_tag_write_without_tagkeys_restriction"
  ],
  "overall_severity": "HIGH",
  "mitre": {
    "tactic": "Credential Access",
    "technique": "Account Manipulation",
    "technique_id": "T1098",
    "sub_technique": "Additional Cloud Credentials",
    "sub_technique_id": "T1098.001",
    "mapping_basis": "HIGH"
  },
  "llm_explanation":
      "The principal arn:aws:iam::344809605381:user/manager_cgid2ztrtw8gc5
       has created an access key using the CreateAccessKey CloudTrail
       event, which is a potential indicator of privilege escalation
       via additional cloud credentials. This action is also supported
       by Phase 1 findings, including
       iam_policy_attached_only_to_group_or_roles and
       iam_tag_write_without_tagkeys_restriction.",
  "llm_priority_justification":
      "High priority due to the creation of an access key, which can be
       used for persistent access to a tenant.",
  "llm_false_positive_assessment":
      "Unlikely false positive as the evidence suggests intentional
       action by the principal.",
```

```
  "llm_remediation_steps":
      "Review IAM policies attached to manager_cgid2ztrtw8gc5 and ensure
       that they are properly restricted. Consider removing direct
       policy attachments and using groups or roles instead.",
  "llm_backend_used": "llama-3-1-8b-instruct"
}
```

Figure 26: Phase Three Large-Language-Model-augmented alert on the iam_privesc_by_key_rotation headline event.

The Large Language Model output correctly identifies the privilege-escalation primitive, references both Phase One finding identifiers, classifies the alert as not a likely false positive, and provides remediation guidance specifically naming the offending principal. The output passes the deterministic citation grounding guard on every field except the MITRE-tactic-confirmed field, where the Large Language Model returned the conservative INSUFFICIENT EVIDENCE marker rather than confirming the deterministic mapping.

The verdict on this scenario is a Pass. Phase One detects the vulnerability class via the V3 detector before any attack runs. Phase Two is silent by design because the privileged target is not admin-equivalent. Phase Three correlates the runtime activity with the static finding and produces eight ACTIVE_EXPLOITATION alerts with the correct MITRE technique mapping. The combined detection demonstrates the framework's complementary-detector design: the vulnerability-class layer catches the static primitive, and the dynamic correlation layer catches the runtime exploitation, even when the breadth-first search returns no path.

5-6 Threats to Validity

The evaluation is honest about its limits. The synthetic-target corpora used in this chapter - IAM Vulnerable, the CloudGoat scenarios, and the synthetic Pathfinding fixtures - are pedagogical environments that do not match the policy density and inherited baselines of a production Amazon Web Services account. The framework's evaluation against a production-density account is identified as future work in Chapter Six.

The Phase One coverage comparison was pre-registered to include both Prowler and Cloudsplaining as external comparators, with a Bonferroni-corrected per-test alpha at k equal to two. Prowler installation on the evaluation host could not be completed within the evaluation timeline because the Prowler-cloud distribution bundles every cloud-provider software-development kit (Amazon Web Services, Azure, Google Cloud, Microsoft Graph, Alibaba Cloud) in a single dependency tree whose total size exceeds two hundred packages and whose deepest installed paths exceed two hundred and sixty characters. The first attempt failed under Microsoft Windows' default maximum-path policy; long-path support was subsequently enabled at the host registry but the installation thereafter remained in a metadata-collection phase for an extended period without progressing to package installation, which made the Prowler scan infeasible within the day. Phase One Metric B therefore rests on the Cloudsplaining comparison alone, with k equal to one and the conventional alpha at zero point zero five. The omission is recorded here so that a reviewer can replicate the comparison on a host where Prowler installs cleanly; the comparator harness retains a Prowler input path so the contingency table can be re-computed when that file becomes available.

Phase Two's breadth-first reachability solver is restricted to admin-equivalent destinations by design. Lateral access to a single sensitive resource - such as the cg_secretsmanager role in scenario two - is therefore not surfaced by the Phase Two output. The framework compensates for this design choice through the vulnerability-class detectors implemented in Phase One and through the runtime correlation in Phase Three; the limitation is documented here for completeness.

The Large Language Model used in Phase Three is a quantized eight-billion-parameter model served locally through a quantized GUFF artifact. Lower-precision weights may reduce output quality compared with full-precision inference. The result is interpreted as a lower bound on the quality of an instruction-following Large Language Model at this scale, and the quantization level is recorded in Module Four of the report. The grounding guard validates structural fidelity of the Large Language Model's output - that the model references only entities present in the evidence bundle and does not assert MITRE labels inconsistent with the deterministic mapping - but does not measure the human-readability or operational utility of the model's explanations and remediation guidance. A user study of the Large Language Model output is identified as future work in Chapter Six.

Chapter Six
Conclusion

6-1 Summary of Contributions

This work has presented Cloud IAM Sentinel, a hybrid Identity-and-Access-Management security analysis framework for Amazon Web Services that combines static posture analysis, privilege-escalation graph reachability, and dynamic CloudTrail correlation with grounded Large-Language-Model reasoning. The framework's principal contributions are four. The first contribution is a static analyzer that evaluates Identity-and-Access-Management Condition operators as edge guards in the privilege-escalation graph and that detects the attribute-based access-control vulnerability classes documented by the National Institute of Standards and Technology and by Amazon Web Services in their tag-based access-control documentation. The second contribution is a tag-mutation reachability extension that models the iam:TagUser, iam:TagRole, and analogous tag-write actions as state-transition edges in a separate graph, allowing the breadth-first search to discover privilege-escalation paths whose feasibility depends on the attacker first mutating an attribute and only then satisfying a previously unsatisfied Condition. The third contribution is a deterministic MITRE ATT&CK Cloud mapping for one hundred and one Identity-and-Access-Management-relevant CloudTrail events, supplied as a static catalog and applied at correlation time rather than asked of the Large Language Model. The fourth contribution is a deterministic citation-grounding guard that rejects Large-Language-Model output whose factual claims are not entailed by the retrieved evidence bundle, providing a verifiable safety property that distinguishes the framework's grounded reasoning from a free-form Large Language Model assistant.

6-2 Summary of Results

The evaluation reported in Chapter Five establishes the framework's performance on three independent corpora and on two end-to-end attack scenarios. Phase One achieves zero incorrect verdicts on the fourteen pre-committed (principal, rule) pairs of the BishopFox IAM Vulnerable adversarial principal set, with three acceptable over-approximations explicitly recognized as documented static-analysis limits. Phase One additionally produces a paired-significantly different per-(check, principal) detection vector from Cloudsplaining at the conventional alpha of zero point zero five. Phase Two detects forty-eight of sixty-six paths under the loose criterion and forty-five of sixty-six paths under the strict criterion on the Datadog Pathfinding.cloud catalog at upstream commit f7170aff, substantially exceeding the catalog-attribution counts for PMapper, Cloudsplaining, and Pacu, with all six paired McNemar's exact tests reaching Bonferroni-corrected significance. Phase Two achieves a recall of one hundred percent on the nineteen testable IAM Vulnerable path classes the deployed free-resources subset instantiates and a recall of sixty-four point five percent on the thirty-one-path Bishop Fox two-thousand-and-twenty-one full denominator.

The two CloudGoat scenarios reported in Chapter Five exercise the framework end-to-end. The negative-control scenario produces zero STRONGLY_CORRELATED alerts and zero ACTIVE_EXPLOITATION-typed alerts, satisfying the strict-correlation negative-control criterion. The attribute-based access-control re-tagging scenario is detected through the framework's V3 detector at Phase One and produces eight ACTIVE_EXPLOITATION alerts with the deterministically-correct MITRE ATT&CK sub-technique label at Phase Three, even though Phase Two's breadth-first search returns no path because the privileged target is not admin-equivalent. The complementary-detector design - vulnerability-class layer at Phase One,

reachability layer at Phase Two, runtime-correlation layer at Phase Three - is illustrated by this combined detection.

6-3 Limitations

The evaluation has documented limits beyond those covered in Section 5-6. The Phase Two breadth-first search returns zero paths when the privileged target is not admin-equivalent; lateral movement to a non-admin sensitive resource therefore relies on the vulnerability-class layer or on Phase Three correlation rather than on graph reachability. The static analyzer cannot resolve Identity-and-Access-Management Conditions that depend on runtime context such as the request time, the source Internet Protocol address, or the calling principal's session attributes; these Conditions are evaluated conservatively as may-satisfy with a confidence flag, which may produce over-approximation in detection. The Large Language Model used for grounded reasoning is a quantized eight-billion-parameter model and may produce non-parseable or ungrounded output on a fraction of bundles; the grounding guard correctly rejects ungrounded output, but the resulting fraction of alerts that lack populated Large-Language-Model fields is a limitation of an eight-billion-parameter quantized model under strict-schema prompting. The evaluation account on which the framework was tested contains pedagogical environments rather than production-density Identity-and-Access-Management configuration; the framework's behavior on a production account remains to be demonstrated.

6-4 Future Work

Several directions follow from the present scope and are identified as future work. The first direction is a streaming deployment of Phase Three. The framework's correlator and prompt builder accept any CloudTrail-shaped input. A production deployment would replace the file-based collector with an Amazon EventBridge-driven Lambda adapter, with Identity-and-Access-Management-write events triggering an incremental rebuild of the Phase One and Phase Two outputs and Identity-and-Access-Management-other events flowing directly into Phase Three. The framework's design is stream-compatible by construction; only the collector adapter requires additional engineering, on the order of thirty lines of code.

The second direction is comparison against a domain-specialized cybersecurity Large Language Model. Several candidates have been released as open-source eight-billion-parameter models specifically fine-tuned on cybersecurity corpora. Comparing the framework's general-purpose Large-Language-Model baseline against such domain-specialized models on the same evidence bundles, prompts, and grounding guard, with a paired McNemar's test on grounding-valid outcomes, is a natural extension of the present evaluation. The required harness is in place; only the additional model baseline must be added to the evaluation runs.

The third direction is behavioral validation of the framework's vulnerability-class detectors on production-scale data. The vulnerability-class detectors have been validated under construct validity through unit-test fixtures derived from the documented attack patterns; behavioral validation against an externally-curated catalog of attribute-based access-control-vulnerable production policies is identified as a primary follow-up work and would close the construct-validity-only framing under which the vulnerability-class detectors are presently evaluated.

The fourth direction is a user study of the Large-Language-Model output. The grounding guard validates structural fidelity of the Large Language Model's output - that the model references only entities present in the evidence bundle and does not assert inconsistent MITRE labels - but does not measure the human-readability or operational utility of the model's explanations and remediation guidance. A controlled study with security practitioners as participants, comparing the framework's grounded-LLM output against a non-augmented baseline on triage speed and remediation correctness, is identified as future work. The required infrastructure - the alert files, the Large-Language-Model baseline, and the grounding guard - is already in place.

The fifth direction is extension of the privilege-escalation graph to long-lived service-managed objects. A subset of the Pathfinding.cloud catalog paths, including paths whose mechanism involves modifying a CloudFormation stack, a Systems Manager document, or an Elastic Container Service task definition that already carries a service role, is currently outside the framework's graph because the graph nodes are restricted to Identity-and-Access-Management principals. Extending the graph to service-managed objects would close approximately eighteen of the sixty-six catalog paths whose mechanism is structurally outside the present scope. PMapper implements such service-object collection in a dedicated module, demonstrating feasibility while showing that this is a distinct contribution-axis orthogonal to the attribute-based access-control extensions that are this work's focus.

References

[1] Amazon Web Services, "AWS Identity and Access Management User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/. [Accessed 2026].

[2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[3] V. C. Hu, D. Ferraiolo, R. Kuhn, A. R. Friedman, A. J. Lang, M. M. Cogdell, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.

[4] Amazon Web Services, "IAM JSON Policy Elements: Condition Operators," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_condition_operators.html. [Accessed 2026].

[5] V. C. Hu, D. R. Kuhn, and D. Yaga, "Attribute Considerations for Access Control Systems," NIST Special Publication 800-205, National Institute of Standards and Technology, 2019.

[6] BishopFox, "IAM Vulnerable: A Benchmark of AWS IAM Privilege-Escalation Paths," BishopFox, 2024. [Online]. Available: https://github.com/BishopFox/iam-vulnerable. [Accessed 2026].

[7] Datadog Security Labs, "Pathfinding.cloud: An Open Catalogue of Cloud Privilege-Escalation Paths," Datadog, 2025. [Online]. Available: https://github.com/DataDog/pathfinding.cloud. [Accessed 2026].

[8] Y. Hu, W. Wang, P. Khurana, M. Lentz, and others, "Detecting Privilege Escalation in IAM Policies via Combined Symbolic and Learning-Based Analysis," in Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2023.

[9] Amazon Web Services, "IAM Tutorial: Define Permissions to Access AWS Resources Based on Tags," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_attribute-based-access-control.html. [Accessed 2026].

[10] Amazon Web Services, "AWS CloudTrail User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/. [Accessed 2026].

[11] The MITRE Corporation, "MITRE ATT&CK for Cloud Matrix (IaaS)," MITRE ATT&CK Knowledge Base, 2024. [Online]. Available: https://attack.mitre.org/matrices/enterprise/cloud/iaas/. [Accessed 2026].

[12] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark, v3.0.0," CIS, 2024.

[13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[14] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[15] Joint Task Force, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53 Revision 5, National Institute of Standards and Technology, 2020.

[16] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, National Institute of Standards and Technology, 2011.

[17] Amazon Web Services, "AWS Shared Responsibility Model," AWS, 2024. [Online]. Available: https://aws.amazon.com/compliance/shared-responsibility-model/. [Accessed 2026].

[18] Y. Wang and others, "Edge-enabled IAM for IoTs with edge-based access management and context-driven sync service," Journal of Systems Architecture, Elsevier, 2024.

[19] Y. Hu, S. Khurshid, and M. Tiwari, "Efficient IAM Greybox Penetration Testing," arXiv preprint arXiv:2304.14540, version 7, 2026.

[20] R. Marbel, Y. Cohen, R. Dubin, A. Dvir, and C. Hajaj, "Cloudy with a Chance of Anomalies: Dynamic Graph Neural Network for Early Detection of Cloud Services' User Anomalies," 2024.

[21] G. Adediran, K. Awuson-David, and Y. Ahmed, "Retrieval-Augmented Large Language Model for AWS Cloud Threat Detection and Modelling: CloudTrail MITRE ATT&CK Mapping," Computers, Materials & Continua, Tech Science Press, 2026, doi: 10.32604/cmc.2026.077606.

[22] A. K. Kaleru, "Identity and access management: foundations, principles, and best practices," World Journal of Advanced Engineering Technology and Sciences, vol. 15, no. 1, pp. 1894-1904, 2025. DOI: 10.30574/wjaets.2025.15.1.0393.

[23] P. Jain, "Identity and Access Management in the Cloud," International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 11, no. 2, pp. 1528-1535, 2025. DOI: 10.32628/CSEIT25112523.

[24] S. Bello and others, "Scalable IAM Audit Framework for Distributed Cloud Security," ResearchGate publication 390928990, April 2025.

[25] S. Bello and others, "A Zero Trust IAM Framework for Unified Auditing in Hybrid Distributed Systems," ResearchGate publication 390929151, April 2025.

[26] V. T. Madireddy, "Graph Neural Network-Based Adaptive Threat Detection for Cloud Identity and Access Management Logs," arXiv preprint arXiv:2512.10280, 2025.

[27] Dikshant and G. Verma, "Alert or Noise? Reducing False Positives with Active Behavioral Analysis for Cloud Security," 2025.

[28] M. H. Nguyen, A. M. Ho, and B. S. To, "SkyEye: When Your Vision Reaches Beyond IAM Boundary Scope in AWS Cloud," arXiv preprint arXiv:2507.21094, 2025.

[29] Cloud Security Alliance, "Top Threats to Cloud Computing 2025," CSA, 2025. [Online]. Available: https://cloudsecurityalliance.org/research/topics/top-threats. [Accessed 2026].

[30] Microsoft Security, "2024 State of Multicloud Security Report," Microsoft, 2024. [Online]. Available: https://www.microsoft.com/en-us/security/business/security-insider/. [Accessed 2026].